

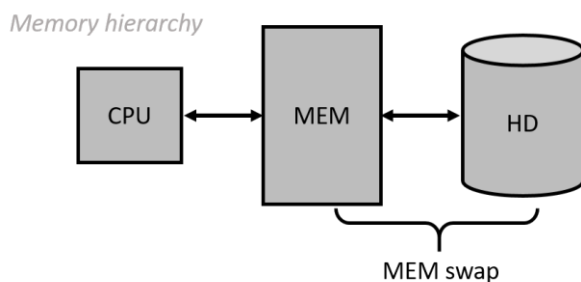
CH6、Memory

容量大且速度快的記憶體

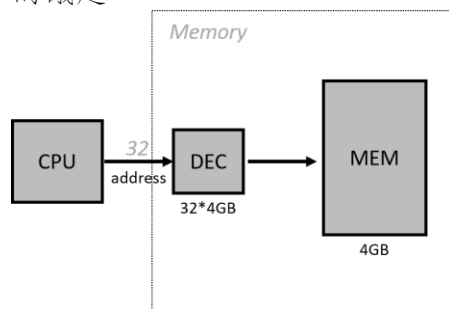
重點一：區域性原則

Memory 的大小：

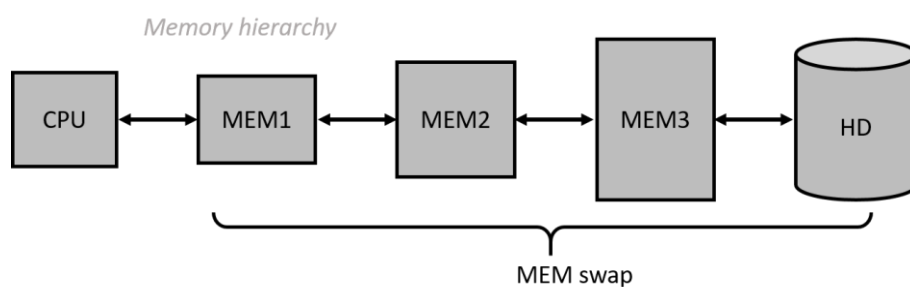
CPU 要執行指令與資料的運算，會需要先從 Hard Disk 中 SWAP 至 Memory，再從 Memory load 到 Register 中。當 Memory 需要空間時，會將沒有用到的資料轉存回 HD，Memory 與 Hard Disk 之間的轉存，稱為 Memory SWAP



Memory 與 HD 的速度差幾千萬倍，所以要盡量降低 SWAP 的次數，擁有較大的 Memory 可改善此問題(Memory 愈大愈好)。然而，Memory 太大也有其缺點：Memory 中還有解碼器(Decoder)來尋找存取位置，因此若 Memory 太大，解碼的時間就需要很長，與速度快矛盾，因此合適的 Memory 設計成為了重要的議題。



因此，現代計算機設計出階層式記憶體 Memory Hierarchy，以達到容量大、且速度快的要求(愈靠近 CPU 愈小、愈遠離 CPU 愈大，且不可跳過存取)：



區域性(Locality)：程式在任何一個時間點，只會存取一小部分的位址空間，分為時間區域性(Temporal Locality)、與空間區域性(Spatial Locality)

1. **Temporal Locality**：當一個資料被存取時，它很可能在短時間內再被存取一次(*If an item is referenced, it will tend to be referenced again soon.*)。例：迴圈
2. **Spatial Locality**：當一個資料被存取時，它周圍的資料也很可能在短時間內被存取(*If an item is referenced, items whose addresses are close by will tend to be referenced soon.*)。例：陣列

例(29)：Explain the following items in a memory system.

1. Locality of reference.
2. Spatial locality.
3. Temporal locality

1. *Locality of reference*：程式在任何一個時間點，只會存取一小部分的位址空間稱為區域性
2. *Spatial locality*：If an item is referenced, items whose addresses are close by will tend to be referenced soon.
3. *Temporal locality*：If an item is referenced, it will tend to be referenced again soon.

例(5)：Which of the following statements are correct?

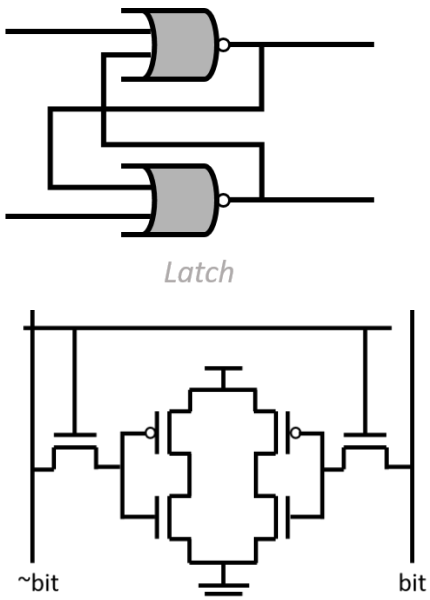
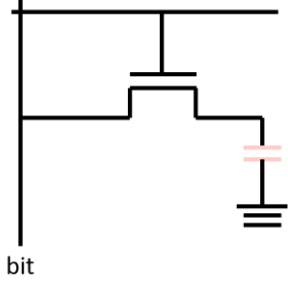
1. For a fixed-size cache, miss rate will decrease as the block size increase.
2. Increasing block size can decrease compulsory miss.
3. For a two-level cache, the design of the primary (first0level) cache focuses on minimizing its miss rate, and the design of the second-level cache focuses on minimizing its hit time.
4. Fetching instructions in a loop utilizes temporal locality while sequential instruction access demonstrates spatial locality.

2、4

例(97)：Write a C program which exhibits the temporal and spatial localities. The C program cannot exceed 5 lines.

```
Clear1(int array[], int size)
{
    int i;
    for(i=0; i<size; i+=1)
        Array[i]=0;
}
```

不同的記憶體技術

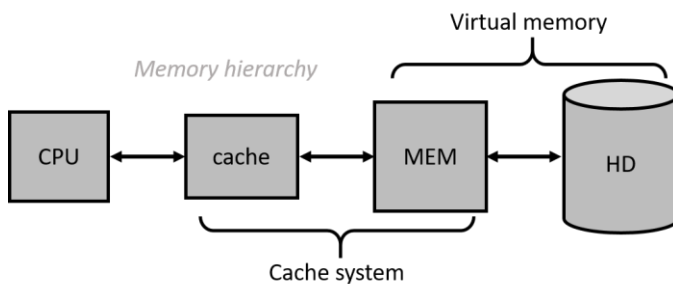
SRAM	DRAM
一位元的資料儲存在一對反向閘之間，雖然速度快，但相對 DRAM 而言，需較大空間(因為其組成為 4-6 個電晶體)	一位元的資料是以電荷形式儲存在電容中(資料需 refresh)，體積小但速度較 SRAM 慢(5-10 倍)
 Latch	 bit

DRAM 雖然便宜，但速度較慢，因為寄生電容充放電時間較長

Memory technology	Typical access time	# per GB in 2008
SRAM	0.5-2.5ns	\$2000-\$5000
DRAM	50-70ns	\$20-\$75
Magnetic Disk	5000000-20000000ns	\$0.2-\$2

重點二：記憶體階層

記憶體階層的目的在於合理成本的建立快速存取記憶體：離 CPU 愈近，使用速度愈快、單價較高的小容量記憶體；離 CPU 愈遠，使用速度愈慢、單價較低的大容量記憶體(CPU、SRAM、DRAM、Magnetic Disk)



例(72)：A storage cell in a DRAM:

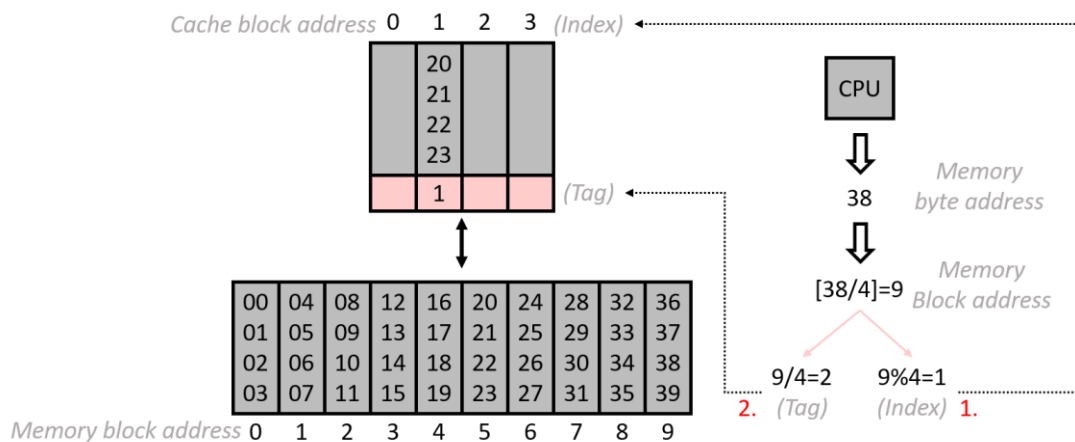
1. is volatile
2. is a capacitor
3. is cheaper than cells in a SRAM
4. is a latch
5. must be refreshed regularly
6. is smaller than cells in a SRAM
7. is typically used for cache RAM
8. is faster than an SRAM

1、2、3、5、6

當 CPU 可在 Cache 中找到資料，稱之為 hit；反之找不到稱之為 miss。兩者分別有其 hit rate 與 miss rate。尋找且找到所花費的時間為 hit time；尋找但找不到、暫停 CPU、並且從 Memory 中搬上找到資料至 Cache 的時間稱為 miss penalty。而搬運、查找、傳輸資料的單位為：Block 或 Line

重點三：直接對映快取記憶體

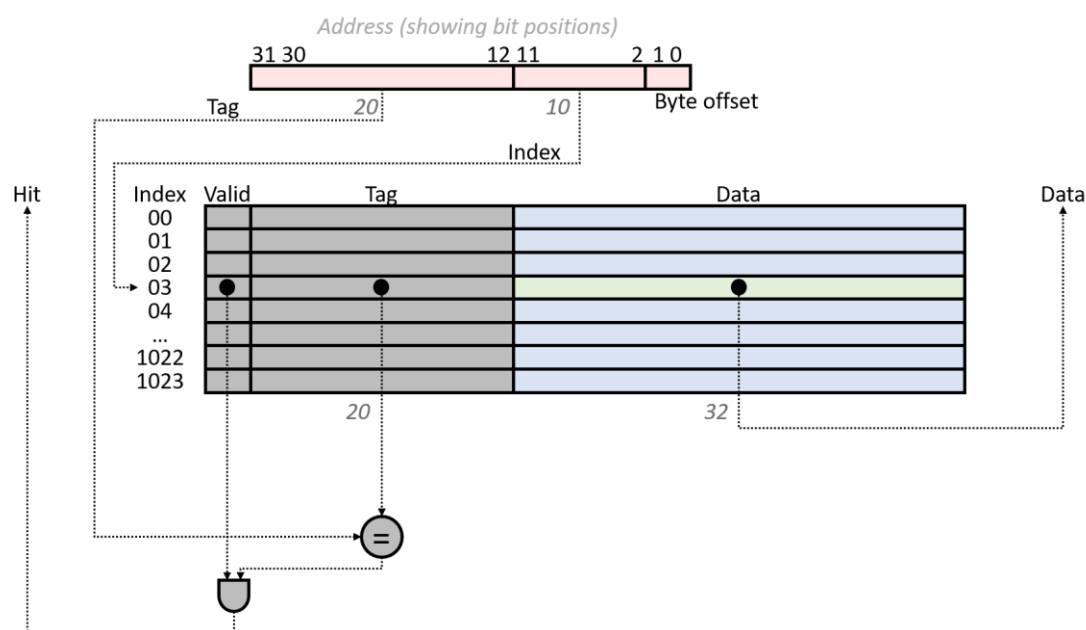
假設 CPU 一次存取一個字組、而且一個區塊也只包含一個字組。為了增加 Cache 與 Memory 對應速度，最簡單的方式就是依照字組在 Memory 中的位址來安排它在 Cache 的位置，此種方式稱為直接對映(Direct Mapped)。



當要找尋一個資料時，會是以 Memory byte address 來表示，但因為資料的存取是以 Block(1 個 Word)為單位，故要先換成 Memory block address($38/4=9$)。接著，因為 CPU 只會存取離它最近的 Cache，故 Memory 的分割會以 Cache 的大小為單位，因此從 Memory 找資料放至 Cache 的位子，也需放至對應位置，1.Index 便因此而產生($9\%4=1$)，而確認 Cache 資料是否正確的，則用 2.Tag 來表示($9/4=2$)

而當以上述流程方式至 Cache 找尋資料有找到，所花時間便是 hit time；反之若沒找到，且從 Memory 搬運正確資料至 Cache 的正確位置，稱為 miss penalty

Cache 的結構圖：



題目會給的資料：

1. Cache size
2. Block size
3. Address length

計算：

1. # of blocks in cache = (1) / (2)
2. 切位置，求 Tag bits
3. $(1 + \text{Tag bit} + \text{block bits}) * \# \text{ of blocks}$

練習：假設記憶體位址為 32 位元，一個可存 64KB 資料量的直接對映快取(且一個區塊存一個字組)的大小為何？

64KB=16K 字組(words)，也就是 2^{14} 個字組，而一個區塊只包含一個字組，所以共有 2^{14} 個區塊。每個區塊有 32 位元的資料、加上標籤位元為 $(32-14-2)$ 及一個有效位元。因此快取大小為：
 $2^{14} * (1 + (32-14-2) + 32) = 2^{14} * 49 = 784Kbits$

練習：在一個資料量大小為 16KB，且每個區塊為 4 個字組的直接對映快取中，假設位址為 32 位元，則此快取總共需要多少個位元？

16KB=4K words，且區塊大小為 4 個 words(2^2)，所以快取總共有 2^{10} 個區塊。每個資料區塊有 $4 * 32 = 128$ 個位元。加上標籤，為 $32-10-4$ 個位元，再加上一個位元的有效位元。因此，此快取的大小為：
 $2^{10} * (1 + (32-10-4) + 128) = 2^{10} * 147 = 147Kbits$

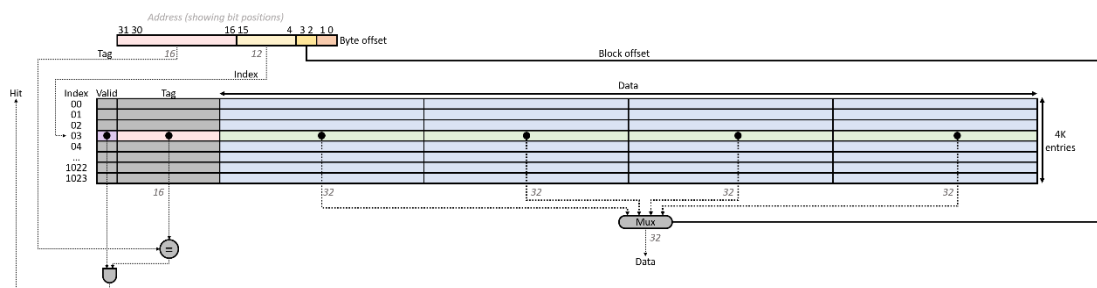
例(8)：Which of the following is (are) true for a cache of 32KB and 32-byte line size?

1. If the cacheable address space is 4GB, the tag width uses 17 bits for a direct-mapped structure.
2. If the cacheable address space is 4GB, the index width uses 16 bits for a direct-mapped structure.
3. If the cacheable address space is 1GB, the index width uses 10 bits for a direct-mapped structure.
4. If the cacheable address space is 1GB, the tag width uses 15 bits for a direct-mapped structure.

1、3、4

採用空間區域性的優點：

每個資料區塊只包含一個字組，並沒有利用到空間區域性的優點。因此，下圖展示了一個存放 **64KB** 資料，且一個資料區塊為 **4** 個字組的快取，即需要一個額外的區塊索引(**block offset**)



Address 分三部分：Tag、Index、Offset(**Block offset** + **Byte offset**)

Tag 用來比對

Block offset = $\log_2(\text{Block Number})$

Entry 數 = $\text{Cache Size} / \text{Block Size} = 2^{\text{Index}}$

Block Number = $\text{Entry} * 2^{\text{block_offset}} = \text{entry} * 2^{\log_2(\text{Block Number})}$

Valid 為是否可用

Cache Size = Entry 數 * **Block size**

整個 cache 大小 = $2^{\text{Index}} * (1 + \text{Tag} + \text{Block Size})$

Tag	Index	Offset	
Block Address		Block Offset = word offset	Byte Offset
Word Address(一個 Address 存一個 Word 的 Data 量)			

練習：Design a 128KB direct-mapped data cache that uses a 32-bit address and 16 bytes per block.

Calculate the following:

- How many bits are used for the byte offset?
- How many bits are used for the set (index) field?
- How many bits are used for the tag?

1. 2 bits

2. $128\text{KB}/16\text{B} = 8\text{K} = 2^{13} \Rightarrow \text{Index field} = 13$

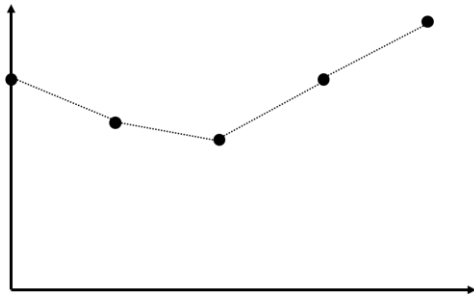
3. $\text{Tag field} = 32 - 13 - 4 = 15 \text{ bits}$

練習：考慮一個有 **64** 個區塊，且一個區塊大小為 **16** 個位元組的快取，位元組位址 **1200** 將被對映到哪一個區塊號碼？

The block is given by (Block address) modulo (Number of cache blocks), where the address of the block is $[\text{Byte_address}/\text{Bytes_per_block}]$. Thus with 16 bytes per block, byte address 1200 is block address $[1200/16] = 75$

區塊大小與快取失誤率：

較大的區塊，利用空間區域性來達到較低的失誤率(增加區塊大小，通常降低了失誤率)；但如果區塊變大，佔快取的比例增加時，快取內的區塊總數會下降，那麼失誤率到最後卻可能會增加，因為許多區塊會相互競爭，導致一個區塊在其含有的字組被存取前，就已經從快取中被置換出去了



只增加區塊大小，將產生一個更嚴重的問題：會增加失誤所造成的成本，也就是失誤懲罰(Miss Penalty)。而抓取區塊的時間可被分為兩個部分：1.拿到第一個字組的延遲時間(Latency)、和 2.將此區塊的剩餘部分傳進來所需的時間。

因此，有以下兩種方式可以減少失誤懲罰時間：

1. 儘早重新開始(Early Restart)：當區塊中所需要的字組送到時，就立刻開始繼續執行，而不是等到整個區塊送過來才開始
2. 需要的字組先送(Requested Word First)、重要的字組先送(Critical Word First)：所需要的字組會先從記憶體送至快取，然後區塊中乘餘的字組再操著被載入(速度『較儘早重新開始』來得快)

練習：For a direct-mapped cache design with 32-bit address, the following bits of the address are used to access the cache.

	Tag	Index	Offset
a	31-10	9-4	3-0
b	31-12	11-5	4-0

1. What is the cache line size (in words)?
2. How many entries does the cache have?
3. What is the ratio between total bits required for such a cache implementation over the data storage bits?

	1	2	3
a	4	64	$(1+22+128)/128 = 1.179$
b	8	128	$(1+20+256)/256 = 1.082$

例(9)：承上題，加入 4、5 題

4. Starting from power on, the following byte-addressed cache references are recorded. How many blocks are replaced?

Address(in Decimal)	Hex	Address(in Decimal)	Hex
0	0x0000 0000	1024	0x0000 0400
4	0x0000 0004	30	0x0000 001E
16	0x0000 0010	140	0x0000 008C
132	0x0000 0084	3100	0x0000 0C1C
232	0x0000 00E8	180	0x0000 00B4
160	0x0000 00A0	2180	0x0000 0884

5. What is the hit ratio?

4. 3 blocks are replaced

Remark: in tag field we ignore the left most 20 0's

Byte address	Tag	Index	Hit/Miss	Result in block replaced
0x0000 0000	00	000000	Miss	
0x0000 0004	00	000000	Hit	
0x0000 0010	00	000001	Miss	
0x0000 0084	00	001000	Miss	
0x0000 00E8	00	001110	Miss	
0x0000 00A0	00	001010	Miss	
0x0000 0400	01	000000	Miss	Yes
0x0000 001E	00	000001	Hit	
0x0000 008C	00	001000	Hit	
0x0000 0C1C	11	000001	Miss	Yes
0x0000 00B4	00	001011	Miss	
0x0000 0884	10	001000	Miss	Yes

5. Hit ratio = $3/12 = 0.25$

例(64) : Suppose we have a direct-mapped cache design with its 32-bit address

- What is the cache line size (in words)?
- How many entries does the cache have?
- What is the ratio between total bits required for such a cache references are recorded: 0, 16, 132, 232, 160, 1024, 30, 140, 3100, 180, 2180
- List the final state of the cache, with each valid entry represented record of <index, tag, data>

- The cache line size = $22 = 4\text{words}$.
- The number of entries = $26 = 64$
- The ratio is $(1+22+128)/128 = 1.179$
-

Byte address	Block address	Tag	Index	Hit/Miss	Byte address	Block address	Tag	Index	Hit/Miss
0	0	0	0	Miss	1024	64	1	0	Miss
4	0	0	0	Hit	30	1	0	1	Hit
16	1	0	1	Miss	140	8	0	8	Hit
132	8	0	8	Miss	3100	193	3	1	Miss
232	14	0	14	Miss	180	11	0	11	Miss
160	10	0	10	Miss	2180	136	2	8	Miss

重點四：快取的基礎概念

Cache miss 的解決方式：

1. Stall CPU，凍結暫存器的內容值
2. 使用一個『分離的控制器』，將記憶體的资料載至 Cache
3. 重新開始

快取寫入：若記憶體資料與快取不同，稱為 Memory Inconsistence

1. Write through：寫入快取的同時也寫入記憶體中
優點：實作簡單
缺點：效能差
2. Write buffer：寫入一緩衝器，若已滿則一樣需暫停
3. Write back：只寫入快取，等到被置換時才寫回記憶體
優點：效能佳
缺點：設計複雜

寫入配置：寫入失誤的處理方法

1. 寫入配置 Write Allocate：在快取中配置一區塊，並由記憶體取得所需區塊資料後，將其寫入配置的區塊中
2. 寫入不配置 No Write Allocate、繞過寫入 Write around：直接寫入記憶體，而不在快取中配置此區塊。

Write Through 搭配 Write Around；Write Back 搭配 Write Allocate；Write Through Cache 與 Write Back Cache 有相同的 Miss Rate

例(38)：Which of the following is (are) true?

1. Write-allocate policy means when a cache write miss occurs. The written data are updated in the next level of memory.
2. There is no cache coherency problem for the write-through cache since the data are written into the next level of memory, and thus the data are consistent between the two caches.
3. A cache can be a shared cache or a private cache.
4. A cache can use either virtual address or physical address for its tag.

3、4

例(24)：Explain the following terms in detail.

1. Write back cache and Write Through cache.
Virtual Memory, Page table, and TLB (Translation Look-aside Buffer).
1. Write back cache : a scheme that handles writes by updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced.
Write through cache : a scheme in which writes always update both the cache and the memory, ensuring that data is always consistent between the two.
2. Virtual Memory : a memory system that uses main memory as a “cache” for secondary storage
Page table : a page table is the table which contains the virtual to physical address translation in virtual memory system
TLB : a cache that keeps track of recently used address mappings to avoid an access to the page table.

分離式快取

結合式快取通常較分離式快取，有較佳的命中率；但使用指令與資料分離的快取，可獲得較大的頻寬。而如果增加較大的頻寬，可以減少失誤的代價，故一般商用機器常使用分離式快取

例(17)：Using separated instruction cache and data cache instead of a unified cache could mainly reduce which kind of hazard?

1. Control hazard
2. Structural hazard
3. Data hazard
4. None of above

2

例(30)：Which of the following statements about caches are not correct?

1. A one-way set associative cache performs the same as a direct-mapped cache.
2. Split cache applies parallel caches to improve cache speed.
3. Translation look-aside buffer (TLB) is a cache for page table and can help accessing virtual memory faster.
4. Write-through caches have better consistency between main memory and cache.
5. All of the above are correct.

2、3、5

註(2)：Split cache is applied to improve cache bandwidth

註(3)：TLB can help accessing physical (main) memory faster.

例(116)：

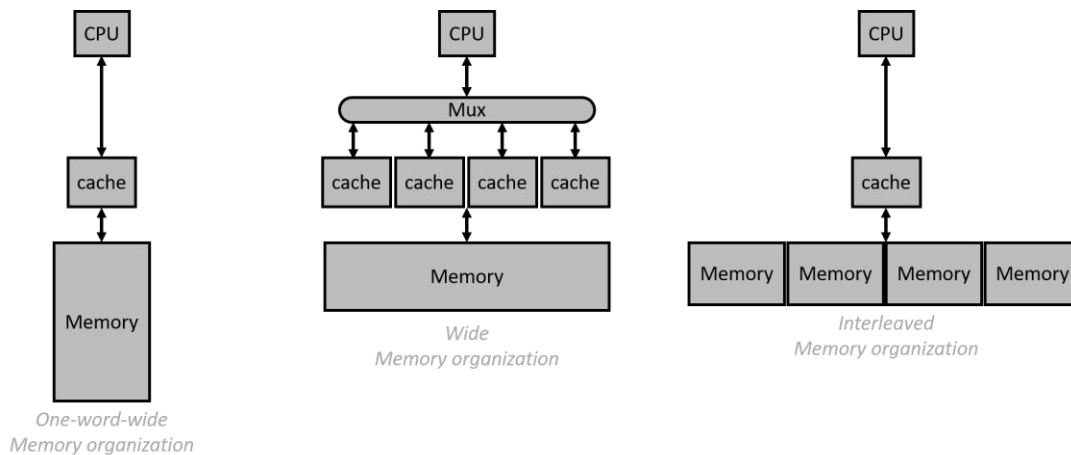
1. Describe at least one clear advantage a Harvard architecture (separate instruction and data caches) has over unified cache architecture (a single cache memory array accessed by a processor to retrieve both instruction and data).
2. Describe one clear advantage a unified cache architecture has over the Harvard architecture.

1. Cache bandwidth is higher for a Harvard architecture than a unified cache architecture.
2. Hit ratio is higher for a unified cache architecture than a Harvard architecture.

因為 miss 的時候，正確資料搬多久、CPU 就停多久，因此減少 miss penalty 的兩種方式有：

1. 一次多搬一些資料：設計支援快取的記憶體系統
2. 搬資料的速度加快：提昇記憶體結構支援快取

設計支援快取的記憶體系統



練習：假設一個快取區塊大小為 4 個字組及寬度為一個字組的 DRAM，以下假設一組記憶體存取時間：

- 送出位址需要 1 個時脈週期
- 起始每個 DRAM 存取的時間為 15 個時脈週期
- 送出一字組資料時間為 1 個時脈週期

分別以上述三種記憶體系統設計，計算由記憶傳送一個區塊至快取記憶體所需要時間。

1. One-word-wide memory organization: $1 + 4 \times 15 + 4 \times 1 = 65 \text{ clock cycles} ; (1 + (15 + 1) \times 4)$
2. Wide memory organization (two-word): $1 + 2 \times 15 + 2 \times 1 = 33 \text{ clock cycles} ; (1 + (15 + 1) \times 2)$
3. Interleaved memory organization (four banks): $1 + 1 \times 15 + 4 \times 1 = 20 \text{ clock cycles} ; (1 + 15 + 1 \times 4)$

例(41)：Assume 1 memory bus clock cycle to send the address, 15 memory bus clock cycles for each DRAM access initiated, 1 memory bus clock cycle to send a word of data. If we have a cache block of four words and a one-word-wide bank of DRAMs, what is the miss penalty?

$$\text{Miss penalty} = 1 + (15 + 1) \times 4 = 65 \text{ clocks}$$

例(54)：Assume that the processor system consists of the microprocessor, cache, bus and main memory, running at 800MHz. A cache block has 32 words and one word has 4 bytes. Given the memory access parameters as follows;

- 1 memory bus clock cycle to send the address;
- 20 memory bus clock cycles for each memory access initiated;
- 1 memory bus clock cycle to send a word of data.

Consider three typical memory system, i.e., 1. One-word-wide memory organization, 2. Wide memory organization with main memory width of four words and 3. Interleaved memory organization with four memory banks.

1. Which memory organization has the smallest number of bytes transferred per bus clock cycle? And what is the number.
2. Suppose that the processor with a 32-word block size has an effective miss rate per instruction of 0.5%. Assume that the CPI without cache misses is 1.5. Which memory organization has the largest speedup when compared with the one-word-wide memory organization? And what is the speedup?

- One-word-wide memory:*

$\text{Clock to send a block} = 1 + (20+1) \times 32 = 673$

$\text{Bytes per clock cycle} = (32 \times 4) / 673 = 0.19$

Four-word-wide memory:

$\text{Clock to send a block} = 1 + (20+1) \times 8 = 169$

$\text{Bytes per clock cycle} = (32 \times 4) / 169 = 0.76$

Interleaved memory:

$\text{Clock to send a block} = 1 + (20+1 \times 4) \times 8 = 193$

$\text{Bytes per clock cycle} = (32 \times 4) / 193 = 0.66$

One-word-wide memory has the smallest number of bytes transferred per bus clock cycle and is 0.19.
- $\text{Effective CPI for one-word-wide memory} = 1.5 + 0.005 \times 673 = 4.865$

$\text{Effective CPI for four-word-wide memory} = 1.5 + 0.005 \times 169 = 2.345$

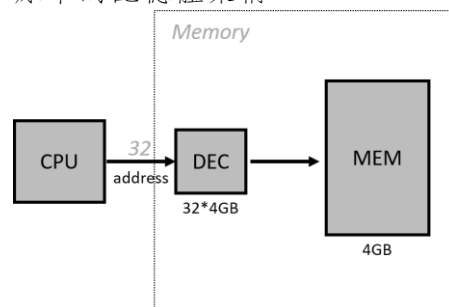
$\text{Speedup} = 4.865 / 2.345 = 2.07$

$\text{Effective CPI for interleaved memory} = 1.5 + 0.005 \times 193 = 2.465$

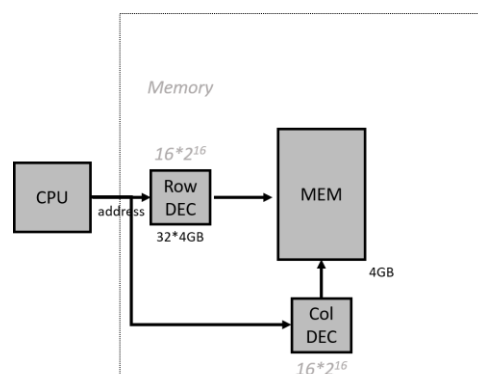
$\text{Speedup} = 4.865 / 2.465 = 1.97$

Four-word-wide memory has the largest speedup compared with the one-word-wide memory and is 2.07

提昇記憶體結構支援快取
原本的記憶體架構：

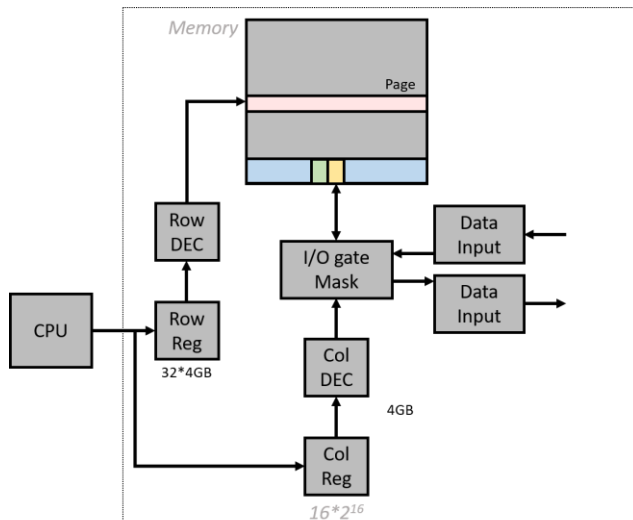


將 Decoder 切分為 Row 與 Column 兩列，可以大大降低解碼時間：

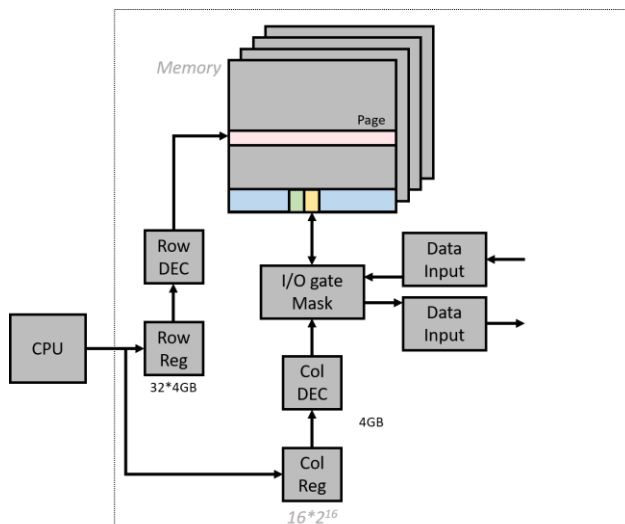


EDO RAM：

Column 部分使用 Mask，便只需要解碼 2 次(一次 Row 一次 Column)，且 Column 為橫向待在同一 Row 解碼即可



SDRAM 即多加好幾個橫切面，變成立體的，並以 Clock 來同步取決：
(Synchronous DRAM)



因為上述方式在 Clock 同步後不用再行解碼，故可以在短時間內存取非常大量的資料，故稱之為暴量存取(Burst Access)

DDR SDRAM：以 Clock 的上源觸發、與下源觸發，都可以做為判斷，故又再 double 存取量(不過寫的時候無法寫、寫的時候無法讀)

QDR SDRAM：改善了 DDR SDRAM 讀時無法寫、寫時無法讀之缺點，故又再 double(整合了 Input 與 Output)

練習：The speed of the memory system affects the designer's decision on the size of the cache block.

Which of the following cache designer guidelines are generally valid?

1. The shorter the memory latency, the smaller the cache block.
2. The shorter the memory latency, the larger the cache block.
3. The higher the memory latency, the smaller the cache block.
4. The higher the memory latency, the larger the cache block.

1 and 4. A lower miss penalty can enable smaller blocks, since you don't have that much latency to amortize, yet higher memory bandwidth usually leads to larger blocks, since the miss penalty is only slightly larger.

練習：Cache block size B can affect both miss rate and miss latency. Assuming the following miss rate table, assuming a 1-CPI machine with an average of 1.35 references (both instruction and data) per instruction, find the optimal block size given the following miss rates for various block sizes.

	8	16	32	64	128
a	8%	3%	1.8%	1.5%	2%
b	4%	4%	3%	1.5%	2%

1. What's the optimal block size for a miss latency of $20 \cdot B$ cycles?
2. What's the optimal block size for a miss latency of $24 \cdot B$ cycles?
3. For constant miss latency, what's the optimal block size?

	1	2	3
a	16-byte	32-byte	64-byte
b	8-byte	8-byte	64-byte

練習：Media applications that play audio or video files are part of a class of workloads called "streaming" workloads; i.e., they bring in large amount of data but do not reuse much of it. Consider a video streaming workload that accesses a 512KB working set sequentially with the following address stream: 0, 4, 8, 12, 16, 20, 24, 28, 32...

1. Assume a 64KB direct-mapped cache with a 32-byte line. What is the miss rate for the address stream above. How is this miss rate sensitive to the size of the cache or the working set? How would you categorize the misses this workload is experiencing, based on the 3C model.
2. Re-compute the miss rate when the cache line size is 16 bytes, 64 bytes and 128 bytes? What kind of locality is this workload exploiting?

1. 12.5% miss rate.

The miss rate doesn't change with cache size or working set. These are cold misses.

2. 25%, 6.25% and 3.125% miss rates for 16-byte, 64-byte and 128-byte blocks. Spatial locality.

重點五：快取效能的量測

1. CPU Time for program

$$= (\text{CPU execution cycles} + \text{Memory Stall cycles}) \cdot \text{Cycle Time}$$

2. Memory Stall Cycles

$$= \# \text{ of Memory access per Program} \cdot \text{Miss Rate} \cdot \text{Miss Penalty}$$

3. Memory Stall Cycles per Instruction

$$= \# \text{ of Memory access per Instruction} \cdot \text{Miss Rate} \cdot \text{Miss Penalty}$$

$$CPI_{\text{eff}} = CPI_{\text{base}} + CPI_{\text{extra}}$$

$$\begin{aligned}
CPI_{\text{extra}} &= \text{Memory Stall extra CPI} + \text{Hazard Stall extra CPI} \\
&= \# \text{ of access per Instruction} * \text{Instruction Cache Miss Rate} * \text{IC Miss Penalty} \\
&\quad + \# \text{ of access per Instruction} * \text{Data Cache Miss Rate} * \text{DC Miss Penalty} \\
&\quad + lw\% * \text{load-use}\% * \text{Penalty} \\
&\quad + beq\% * \text{miss-predict}\% * \text{Penalty} \\
&\quad + j\% * 100\% * \text{Penalty}
\end{aligned}$$

Suppose a program with 100 instruction among while 30 lw/sw	split		combined
	I-C	D-C	Cache
# of access per program	100	30	130
# of access per instruction	1	0.3	1.3

練習：Assume an instruction cache miss rate for a program is 2% and a data cache miss rate is 4%. If a processor has a CPI of 2 without any memory stalls and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Use the instruction frequencies for SPECint2000 (load/store : 36%)

$$CPI_{\text{effected}} = CPI_{\text{base}} + CPI_{\text{extra}}$$

# of access per instruction	I-C Miss Rate	I-C Miss penalty	
1	0.02	100	2
# of access per instruction(lw/sw)	D-C Miss Rate	D-C Miss penalty	
0.36	0.04	100	1.44

$$\text{Instruction cache stall per instruction} = 1 * 2\% * 100 = 2$$

$$\text{Data cache stall per instruction} = 0.36 * 0.04 * 100 = 1.44$$

$$\text{Memory stall per instruction} = 2 + 1.44 = 3.44$$

$$CPI_{\text{effected}} = CPI_{\text{base}} + CPI_{\text{extra}} = 2 + 3.44 = 5.44$$

$$\text{Ratio of the execution time} = 5.44 / 2 = 2.72$$

例(10)：Assume the miss rate of an instruction cache is 1% and the miss rate of the data cache is 3%. If a processor has a CPI of 1 without any memory stalls and the miss penalty is 100 cycles for all misses, determine how much faster a processor would run with a perfect cache that never missed. Assume the frequency of all loads and stores is 30%

$$\text{The CPI considering stalls is } 1 + 1 * 0.01 * 100 + 0.3 * 0.03 * 100 = 2.9$$

$$\text{The processor run with a perfect cache is } 2.9 / 1 = 2.9 \text{ times faster than without a perfect cache.}$$

例(19)：Assume an instruction cache miss rate of 2% and a data cache miss rate of 4% If a machine has a CPI of 2 without any memory stalls and the miss penalty is 40 cycles for all misses, determine how much faster a machine would run with a perfect cache that never missed. Assume 36% of instructions are loads/stores.

$$\text{The CPI considering stalls is } 2 + 1 * 0.02 * 40 + 0.36 * 0.04 * 40 = 3.376$$

$$\text{The processor run with a perfect cache is } 3.376 / 2 = 1.688 \text{ times faster than without a perfect cache.}$$

例(60) : Consider a computer operates with clock frequency 2 GHz. It has the split cache organization. If all memory accesses are hit in the cache, the CPI is 2. The cache miss penalty is 40 ns. Suppose a program is run on this computer. Among the instructions been executed, 25% of them are memory access instructions. The miss rate for instruction cache is 2%, and for data cache I 4%. What is the actual CPI?

$$\text{Miss penalty} = 40\text{ns} * 2\text{GHz} = 80 \text{ clock cycles}$$

$$\text{Actual CPI} = 2 + 1 * 0.02 * 80 + 0.25 * 0.04 * 80 = 4.4$$

例(28) : A computer with a direct0mapped cache has an instruction miss rate of 2% and data miss rate of 1.7%. Suppose the computer has a clock cycle time of 20 ns, and the CPI without memory stall is 2 cycles per instruction. Suppose that the miss penalty is 200 ns, and there is , on average, one memory reference for data every two instructions. How much time, on average, does the computer need to execute a program with 10000 instructions?

$$\text{CPI}_{\text{effect}} = 2 + 1 * 0.02 * (200\text{ns}/20\text{ns}) + 0.5 * 0.017 * (200\text{ns}/20\text{ns}) = 2.285$$

$$\text{Execution time} = 10000 * 2.285 * 20\text{ns} = 457000\text{ns} = 457\text{us}$$

練習 : The data cache has a 92% hit rate and a 2-cycle hit latency, and the miss penalty is 124 cycles. 30% of instructions are loads and stores. The instruction cache has a hit rate of 90% with a miss penalty of 50 cycles. Assume the base CPI using a perfect memory system is 1.0. Calculate the CPI of the pipeline, assuming everything else is working perfectly. Assume the load never stalls a dependent instruction and assume the processor must wait for stores to finish when they miss the cache. Finally, assume that instruction cache misses and data cache misses never occur at the same time. Show your work.

1. Calculate the additional CPI due to the instruction cache stalls
2. Calculate the addition CPI due to the data cache stalls.
3. Calculate the overall CPI for the machine.

$$1. \text{ The addition CPI due to instruction cache stalls} = 1 * 0.1 * 50 = 5$$

$$2. \text{ The addition CPI due to data cache stalls} = 0.3 * 0.08 * 124 = 2.976$$

$$3. \text{ The overall CPI} = 1 + 5 + 2.976 = 8.976$$

例(1) : Calculate the performance of a processor taking into account stalls due to data cache and instruction cache misses. The data cache has a 92% hit rate and a 2-cycle hit latency. Assume that latency to memory and the cache miss penalty together is 100 cycles. The instruction cache has a hit rate of 90% with a miss penalty of 50 cycles. Assume the load never stalls when they miss the cache. Finally, assume that instruction cache misses and data cache misses never occur at the same time.

1. Calculate the average memory access latency for the data cache.
2. Assume the base CPI using a perfect memory system is 1.0. Calculate the additional CPI of the pipeline due to the Instruction cache stalls.
3. Same as 2, but calculate the additional CPI due to the data cache stalls.
4. Assume 30% of instructions are loads and stores. Calculate the overall CPI ofr the machine.

$$1. \text{ AMAT} = 2 + 0.08 * 100 = 10$$

$$2. \text{ The additional CPI due to instruction cache stalls} = 1 * 0.1 * 50 = 5$$

$$3. \text{ Suppose 30\% of instructions are loads and stores. The additional CPI due to data cache stalls} = 0.3 * 0.08 * 100 = 2.4$$

$$4. \text{ Overall CPI} = 1 + 0.3 * 0.08 * 100 + 1 * 0.1 * 50 = 5.7$$

練習：Suppose we increase the performance of the computer in the previous example by doubling its clock rate. Since the main memory speed is unlikely to change, assume that the absolute time to handle a cache miss does not change. How much faster will the computer be with the faster clock, assuming the same miss rate as the previous example?

Measured in faster clock cycle, the new miss penalty will be 200 cycles.

Memory stall per instruction = $2\% \times 200 + 36\% \times (4\% + 200) = 6.88$

Faster system with cache miss, $CPI = 2 + 6.88 = 8.88$

Slower system with cache miss, $CPI = 5.44$

The faster clock system will be

Execution time of slow clock / Execution time of faster clock

$= I \times CPI_{slow} \times Cycle\ time / (I \times CPI_{fast} \times 1/2 \times Cycle\ time) = 5.44 / (8.88 \times 1/2) = 1.23\ times\ faster$

例(106)：Suppose you had a computer that, on average, exhibited the following properties on the program that you run:

Instruction miss rate: 2%

Data miss rate: 4%

Percentage of memory instructions: 30%

Miss penalty: 100 cycles

There is no penalty for a cache hit (i.e. the cache can supply the data as fast as the processor can consume it.) You want to update the computer, and your budget will allow one of the following:

Option 1: Get a new processor that is twice as fast as your current computer.

The new processor's cache is twice as fast too, so it can keep up with the processor.

Option 2: Get a new memory that is twice as fast.

Which is a better choice? And what is the speedup of the chosen design compared to the old machine?

Suppose that the base $CPI = 1$

$CPI_{old} = 1 + 0.02 \times 100 + 0.04 \times 0.3 \times 100 = 4.2$

$CPI_{opt1} = 0.5 + 0.02 \times 100 + 0.04 \times 0.3 \times 100 = 3.7$

$CPI_{opt2} = 1 + 0.02 \times 50 + 0.04 \times 0.3 \times 50 = 2.6$

Option 2 is $4.2/2.6 = 1.62\ times\ faster\ than\ the\ old\ machine.$

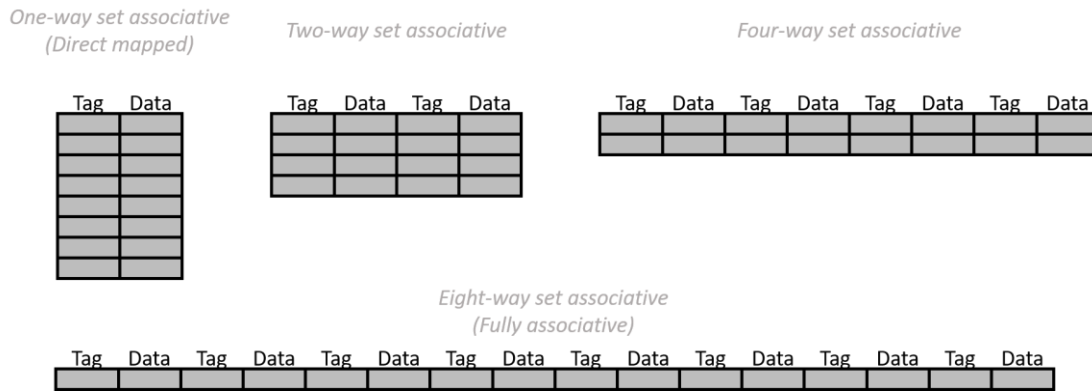
註(Option 1)：processor 快兩倍且 cache 跟得上，表示在不考慮 stall 的情況下，一個指令的執行時間會減半。一個指令的執行時間 = $CPI \times cycle\ time$ 。本題並未假設 clock rate double，所以 cycle time 並未減半，因此 CPI 要減半才能為反應出指令執行時間減半。題目沒特別說明 base CPI 都假設為 1，因此 option 1 的 CPI 要變成 0.5 才可說明 processor 快兩倍且 cache 跟得上的事實。

失誤率與失誤處罰時間是影響快取效能的兩個主因，以下介紹兩種改善方式：

1. 集合關聯式快取(Set Associative Cache)：藉由減少記憶體區塊競爭同一個快取位置的機率(重點六)
2. 多層快取(Multilevel Cache)：在階層中加入額外一層快取，來減少失誤處罰時間(重點七)

重點六：集合關聯式快取

相對於一對一的直接對映快取，集合關聯式快取，是先將快取分成數個集合，每個集合數量固定，並且在對映時，只要集合中還有空出的快取，記憶體區塊就能使用；一個快取包含 n 個區塊，就稱為 n -way 或關聯度(Associativity)。一般來說有四種：



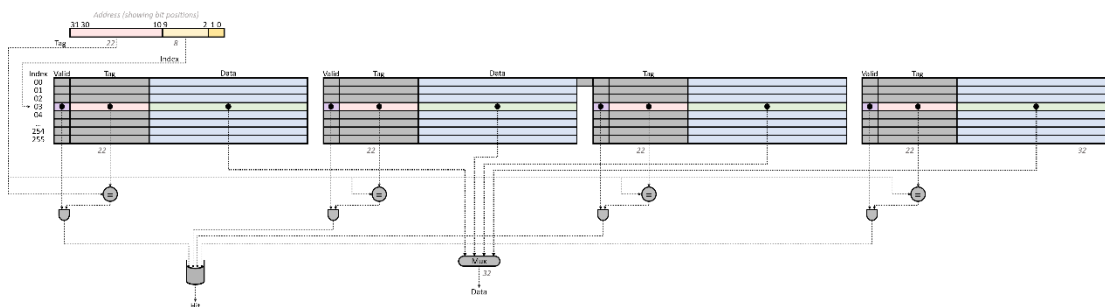
當對映的過程，集合已經滿了，則需要置換。找出需要被置換的區塊。方法又分兩種：1.隨機置換 Random、2.最近最不常用(Least recently used, LRU)

例(6)：Describe two replacing schemes used in set-associative cache.

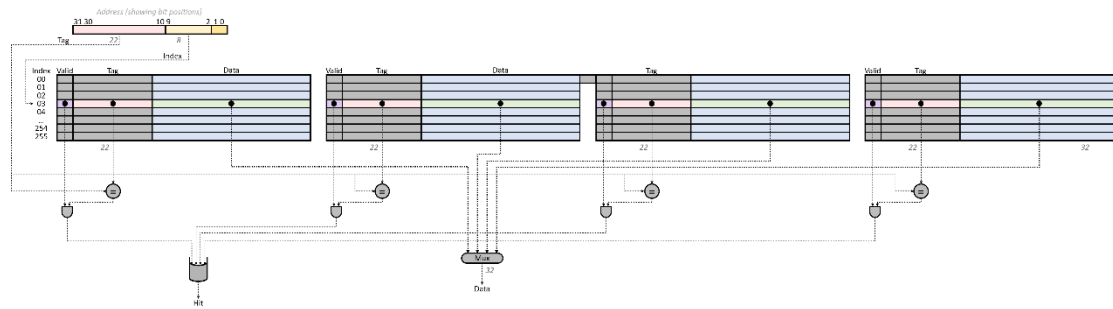
1. 隨機(Random)：被置換的區塊是隨機選取的
2. 最近最不常用(least recently used, LRU)：被置換的區塊是最長時間沒有被使用的

找尋快取中的區塊：

of block in a Cache (64) = # of sets * associativity ($1*64, 2*32, 4*16, 8*8$)



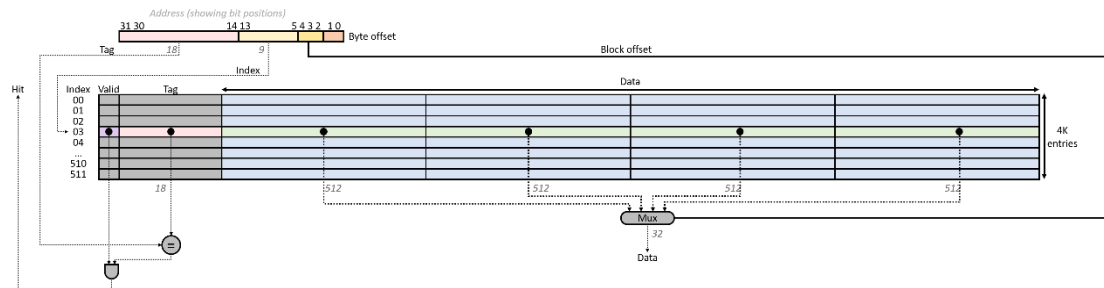
例(50)：Plot the block diagram for 4-way set-associative cache and explain how the address, which is composed of tag, index, and offset, sent from the processor is used to access the cache.



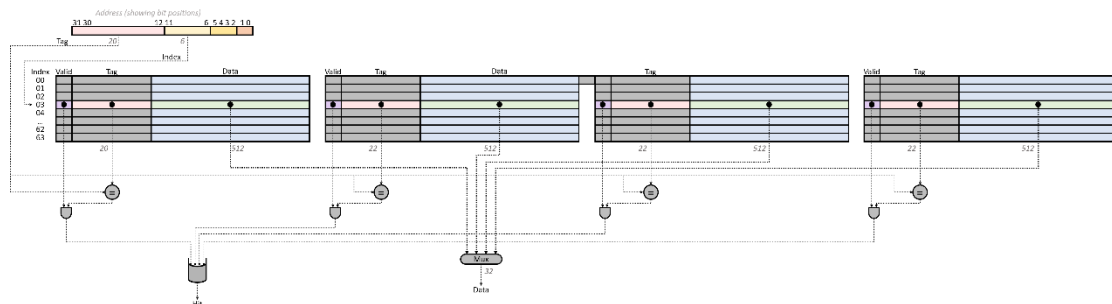
例(63) : Design a cache for the following cases. Show the block diagram of the design, including buses used for the cache, the tag memory, data memory, and how the address to the cache is used. The cache size is 16K-bytes for all cases.

1. A direct-mapped cache with 32-bytes of the line size.
2. A four-way set associative cache with 64-bytes of the line size.
3. Compute the tag memory size in bits of the caches in 1 and 2 respectively.

1.



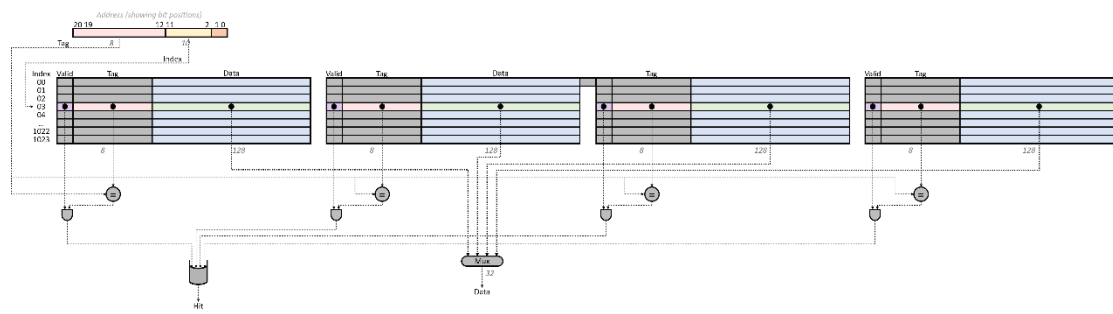
2.



3. Direct-mapped cache: $18 \times 512 = 9216$ bits
Four-way set associative cache: $20 \times 4 \times 64 = 5120$ bits

例(16) : A set associative cache has a block size of four 32-bit words and a sized of 4. The cache can accommodate a total of 16K words. The main memory size that is cacheable is $1M \times 32$ bits. Design the cache structure and show how the processor's addresses are interpreted.

$1M(2^{20}) \times 32\text{bits} \Rightarrow 20 \text{ bits 的 word address}(32\text{bits}) \cdot \text{轉 byte address 要扣掉 2 bits offset}$
 $16K \text{ words} / 4 \text{ words} = 4K \Rightarrow 4K / 4(\text{set size}) = 1K (\text{entry/index})$
 $\text{Tag} = 20 - 2 - 10 = 8$



練習： Assuming a cache of 4K blocks, a four-word block size, and a 32-bit address, find the total number of sets and the total number of tag bits for caches that are direct mapped, two-way and four-way set associative, and fully associative.

4-word (16 byte) per block, total $(32-4) = 28$ bits for tag and index

1. Direct mapped:

Number of sets = 4K, $\log(4K) = 12$, number of tag bits = $(28-12)*4K = 64K$ bits

2. 2-way associative:

Number of sets = $4K/2$, number of tag bits = $(28-11)*2*2K = 34*2K = 68K$ bits

3. 4-way associative:

Number of sets = $4K/4$, number of tag bits = $(28-10)*4*1K = 72*1K = 72K$ bits

4. Fully associative:

Only one set, number of tag bits = $28*4K = 112K$ bits

例(52)： A memory system has 1M words. Each word is an 8-bit byte. The memory is organized into blocks of 8 words each. The cache has 256K words, organized into cache lines of 8 words each. The memory cache is organized into 4-way set associative cache.

- 13 bits are needed for cache set address.
- 4 bits are for Tag
- 21 bits are needed to address all words
- 17 bits are needed to address all blocks

1、2、4

註： address format：

4	13	3
Tag	Index	Block offset

註(3)： should be 20 bits

例(100)： Please answer the following questions about memory hierarchy.

1. What are TLB and page table? Please describe clearly and systematically how a memory access is completed by the processor cache/main memory/TLB/page table/hard disk.
2. A computer system has a cache memory with 128K bytes. The 32-bit memory address format is as follows: Tag bits: 31-15, Index (or Set) bits: 14-4, Offset bits: 3-0. Please derive the number of degrees of set associativity in this cache.

1. TLB: A cache that keeps track of recently used address mappings to avoid an access to the page table. Page table: The table containing the virtual to physical address translations in a virtual memory system.

A virtual address issued from CPU is translated by the TLB. When a TLB miss occurs, the entry of the mapping will move from page table to TLB. If page table can not find this mapping, then a page fault

occurs. Operating System moves the missing page from hard disk to the physical memory and the mapping thus will exist in the page table. We use the translated physical address to search data/instruction in the cache. If cache hit, data/instruction will send to CPU; otherwise, cache miss occurs. A separate control will move the missing block from memory to cache and then send to CPU.

2. Offset = 4 bits $\Rightarrow$ block size = 16 bytes
 The number of blocks in the cache = $128KB/16B = 8K$
 The index field has 11 bits $\Rightarrow$ the cache has 2K sets
 The number of blocks in a set = $8K/2K = 4$
 Hence, the degree of set associativity = 4

例(108) : Given a 2^s -byte, 2^L -byte-per-line cache in an M-bit, byte-addressable memory system,

1. What is the range of index field size, in no. of bits, in a memory address while accessing the cache?
2. Repeat 1 for the tag field size.

The number of blocks in the cache = $2^s/2^L = 2^{s-L}$

	Tag	Index	Offset
Direct-mapped	M-S	S-L	L
Fully associative	M-L	0	L

例(18) : A system that has k-bit address with S bytes of cache memory stores data in B-bytes lines, where $B=2^b$. The cache memory is an A-way set associative cache. Please estimate the number of bits needed to implement the cache in terms of k, S, B, b, and A. (Each cache row entry has 1-bit valid bit, tag bits, and data blocks.)

Answer size: k bits, Cache size: S bytes/cache, Block size: $B=2^b$ bytes/block,

Associativity: A blocks/set

Number of sets in the cache = S/AB ;

Number of bits for index = $\log(S/AB) = \log(S/A) - b$

Number of bits for the tag = $k - (\log(S/A) - b) - b = k - \log(S/A)$

Number of bits needed to implement the cache = sets/cache*associativity*(data + tag + valid)

= $S/AB * A * (8*B + k - \log(S/A) + 1) = S/B * (8*B + k - \log(S/A) + 1)$ bits

練習 : How many total SRAM bits will be required to implement a 256KB four-way set associative cache. The cache is physically-indexed cache, and has 64-byte blocks. Assume that there are 4 extra bits per entry: 1 valid bit, 1 dirty bit, and 2 LRU bits for the replacement policy. Assume that the physical address is 50 bits wide.

The number of sets = $256K/(4*64) = 1024$

Tag bits = $50 - 10 - 6 = 34$

The total number of each block required = $4 + 34 + 64*8 = 550$ bits

The total number of SRAM bits required = $1024*4*550 = 2200$ Kbits

例(21) : True & False Questions

The set index decreases in size as cache associativity is increased (assume cache size and block size remain the same).

True

練習：Design a 8-way set associative cache that has 16 blocks and 32 bytes per block. Assume a 32 bit address. Calculate the following:

1. How many bits are used for the byte offset?
2. How many bits are used for the set (index) field?
3. How many bits are used for the tag?

1. 2 bits
2. 1 bit
3. 26 bits

重點七：使用多層快取來減少失誤處罰時間

多層快取可以有效減少失誤處罰時間。而設計主要快取與次要快取的考量不同：主要快取注重在減少命中時間來縮短時脈週期，並讓次要快取注重在失誤率，以降低存取記憶體花太長時間的影響

練習：Which of the following is generally true about a design with multiple levels of caches?

1. First-level caches are more concerned about hit time, and second-level caches are more concerned about miss rate.
2. First-level caches are more concerned about miss rate, and second-level caches are more concerned about hit time.

- 1.

例(62)：Which of the following on-chip cache hierarchies are often implemented in modern processors?

1. Split L1 data and instruction caches
2. Combined L2 caches
3. Split L2 data and instruction caches
4. A write-through L2 cache

1、2

L1 注重 Hit Time，IC、DC 分開較快；L2 注重 Miss Rate，合在一起較好

L1 使用 Write Through、Write Back 差不多，因為 L2 速度也不慢；L2 則使用 Write Back，因為寫入 Memory 速度差太多

例(95)：True or false:

1. Thrashing occurs if a program constantly accesses more virtual memory than it has physical memory, causing continuously swapping between memory and disk.
2. The idea of using two levels of cache is that 1st level cache is to minimize the cache miss ratio and the 2nd level cache is to reduce the cache hit time.

1. True
2. False, 1st level cache is to minimize the cache hit time and the 2nd level cache is to reduce the cache miss ratio.

練習：Suppose we have a processor with a base CPI of 1.0, assume references hit in the primary cache, and a clock rate of 4GHz. Assume a main memory access time of 100ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 2%. How much access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.5%.

CPU	L1	MM
MRP_i	0.02	-
MP_i	$100ns * 4G = 400$	-

$$1 + 0.02 * 400 = 9$$

CPU	L1	L2	MM
MRP_i	0.02	0.005	-
MP_i	$5ns * 4G = 20$	400	-

$$1 + 0.02 * 20 + 0.005 * 400 = 3.4$$

例(102)：Suppose we have a processor with a base CPI of 1.0, assuming all references hit in the primary cache, and a clock rate of 5 GHz. Assume a main memory access time of 100 ns, including all the miss handling. Suppose the miss rate per instruction at the primary cache is 2%. How much faster will the processor be if we add a secondary cache that has a 5 ns access time for either a hit or a miss and is large enough to reduce the miss rate to main memory to 0.6%?

The miss penalty to main memory is $100/0.2 = 500$ clock cycles.

*For the processor with one level of caching, total CPI = $1.0 + 500 * 2\% = 11.0$.*

The miss penalty for an access to the 2nd level cache is $5/0.2 = 25$ clocks.

*For the two level cache, total CPI = $1.0 + 2\% * 25 + 0.6\% * 500 = 4.5$.*

Thus, the processor with the secondary cache is faster by $11.0/4.5 = 2.44$

例(15)：Assume that a 2-way set-associative cache contains 1024 blocks (lines) and each block has 8 bytes.

1. If the address is 16 bits long, what values are the tag “bits” in the block containing the data with the address “0x8F8A”?
2. Following 1, right after the data with the address 0x8F8A is accessed, what range of data addresses can be accessed in this cache, i.e. hits (because of the locality associated with 0x8F8A)? Express your answer as “0XXXXX-0XXXXX”
3. Assume that the access time to this cache is 2 ns and the main memory access time is 100 ns, including all the miss handling. The miss rate of this cache is 5%. What is the speedup if we add a secondary cache that has a 20 ns access time and the miss rate of this secondary cache is 1%?

1. $0x8F8A = 1000\ 1111\ 1000\ 1010$

Tag	Index	Offset
1000	111110001	010

The value of tag bits is 8.

2. $0x8F88-0x8F8F$

3. Suppose the cache access time determine the CPU clock cycle. i.e., CPU clock cycle = 2ns.

And suppose base CPI = 1

*CPI for one level cache = $1 + 0.05 * 50 = 3.5$*

*CPI for two level cache = $1 + 0.05 * 10 + 0.01 * 50 = 2$*

註(3)：We regard all miss rates as miss rate per instruction

例(85) : Consider a computer has clock rate of 2.5GHz. The CPI with a perfect cache is 3. The cache miss penalty is 40ns. Assume the cache miss rate is 6%.

1. What is the actual CPI?
2. If a second level cache with the 1st level miss penalty 10ns is added, the miss rate to the main memory is reduced to 2%. What is the speedup?

1. $\text{Miss penalty for cache miss} = 40\text{ns} * 2.5\text{GHz} = 100(\text{clocks})$
 $\text{Actual CPI} = 3 + 0.06 * 100 = 9$
2. $\text{Miss penalty for the 1st level miss} = 10\text{ns} * 2.5\text{GHz} = 25 (\text{clocks})$
 $\text{Effective CPI for 2-level cache} = 3 + 0.06 * 25 + 0.02 * 100 = 6.5$
 $\text{Speedup} = 9 / 6.5 = 1.38$

練習 : Consider a processor with the following parameters:

	a	b
Base CPI, no memory stalls	2.0	2.0
Processor Speed	3GHz	1GHz
Main memory access time	125ns	100ns
1 st -level cache miss rate per instruction	5%	4%
Second-level cache, direct-mapped speed	15 cycles	10 cycles
Global miss rate with 2 nd -level cache, direct-mapped	3.0%	4.0%
2 nd -level cache, 8-way set associative speed	25 cycles	20 cycles
Global miss rate with 2 nd -level cache, 8-way set associative	1.8%	1.6%

1. Calculate the CPI for the processor in the table using: 1. Only a first-level cache, 2. A second-level direct-mapped cache, and 3. A second-level eight-way set-associative cache.
2. It is possible to have an even greater cache hierarchy than two levels. Given the processor above with a second-level, direct-mapped cache, a designer wants to add a third-level cache that takes 50 cycles to access and will reduce the global miss rate to 1.3%. Would this provide better performance? In general, what are the advantages and disadvantages of adding a third-level cache?
3. In older processors such as the Intel Pentium or Alpha 21264, the second level of cache was external (located on a different chip) from the main processor and the first-level cache. While this allowed for large second-level caches, the latency to access the cache was much higher, and the bandwidth was typically lower because the second-level cache ran at a lower frequency. Assume a 512KB off-chip second-level cache has a global miss rate of 4%. If each additional 512KB of cache lowered global miss rates by 0.7%, and the cache had a total access time of 50 cycles, how big would the cache have to be to match the performance of the second-level direct-mapped cache listed in the table? Of the eight-way set-associative cache?

1. 表

a	$\text{Memory miss cycles: } 125\text{ns} * 3\text{G} = 375$ 1. $\text{Total CPI: } 2.0 + 375 * 5\% = 20.75$ 2. $\text{Total CPI: } 2.0 + 15 * 5\% + 375 * 3\% = 14$ 3. $\text{Total CPI: } 2.0 + 25 * 5\% + 375 * 1.8\% = 10$
b	$\text{Memory miss cycles: } 100 \text{ clock cycles}$ 1. $\text{Total CPI: } 2.0 + 100 * 0.04\% = 6.0$ 2. $\text{Total CPI: } 2.0 + 100 * 0.04\% + 10 * 0.04 = 6.4$ 3. $\text{Total CPI: } 2.0 + 100 * 0.016\% + 20 * 0.04 = 4.4$

2. 表

a	$\text{Total CPI: } 2.0 + 15 * 5\% + 50 * 3\% + 375 * 1.3\% = 9.125$ This would provide better performance, but may complicate the design of the processor. This could lead to: more complex cache coherency, increased cycle time, larger and more expensive
---	--

	<i>chips.</i>
<i>b</i>	<i>Total CPI: $2.0 + 100*0.013 + 10*0.04 + 50*0.04 = 5.7$ This would provide better performance, but may complicate the design of the processor. This could lead to: more complex cache coherency, increased cycle time, larger and more expensive chips.</i>

3. 表

<i>a</i>	<i>Total CPI: $2.0 + 50*5\% + 375*(4\%-0.7\%*n)$ $n=2 \Rightarrow 1.5\text{MB L2 cache to match direct-map}$ $n=4 \Rightarrow 2.5\text{MB L2 cache to match 8-way}$</i>
<i>b</i>	<i>Total CPI: $2.0 + 50*0.04 + 100*(4\%-0.7\%*n)$ $n=2 \Rightarrow 1.5\text{MB L2 cache to match direct-map}$ $n=5 \Rightarrow 3\text{MB L2 cache to match 8-way}$</i>

註(3)a : Let $2.0 + 50*5\% + 375*(4\%-0.7\%*n) = 14 \Rightarrow n=2.1$

Let $2.0 + 50*5\% + 375*(4\%-0.7\%*n) = 10 \Rightarrow n=3.6$

區域失誤率與全域失誤率

Global miss rate(GMR) : The fraction of references that miss in all levels of a multilevel cache.

Local miss rate(LMR) : The fraction of references to one level of a cache that miss : used in multilevel hierarchies.

CPU	GMR	LMR
L1	60/1000	60/1000
L2	30/1000	30/60
L3	5/1000	5/30

$$\text{GMR}_i = \text{LMR}_1 * \text{LMR}_2 * \dots * \text{LMR}_i$$

平均記憶體存取時間(AMAT)

Hit Time	T_1	T_2		T_n
Miss Rate	M_1	M_2	...	M_n
Miss Penalty	P_1	P_2	...	P_n

$$\text{AMAT} = T_1 + M_1*P_1 + M_2*P_2 + \dots + M_n*P_n = T_1 + \text{Sum } M_i*P_i$$

例(119) : The average memory access time (AMAT) is defined as

$$\text{AMAT} = \text{time for a hit} + \text{miss rate} * \text{miss penalty}$$

Consider the following two machines:

Machine 1: 100MHz, a hit time of 1 clock cycle, a miss rate of 5% and a miss penalty of 20 clock cycles.

Machine 2: 100MHz, a hit time of 1.2 clock cycles, a miss rate of 3% and a miss penalty of 25 clock cycles.

Determine which machine has smaller AMAT.

$$\text{AMAT for Machine 1} = 10\text{ns} * (1 + 0.05 * 20) = 20\text{ns}$$

$$\text{AMAT for Machine 2} = 10\text{ns} * (1.2 + 0.03 * 25) = 19.5\text{ns}$$

Hence, Machine 2 has smaller AMAT.

例(22) : Assume that a processor with a 1ns clock cycle time, a miss penalty of 16 clock cycles, a miss rate of 0.07 misses per instruction, and a cache access time (including hit detection) of 1 clock cycle. Note that the read and write miss penalties are the same and ignore other write stalls. What is the average memory access time per instruction?

$$AMAT \text{ per instruction} = (1 + 0.07 * 16) * 1ns = 2.12ns$$

練習 : Suppose that in 1000 memory references there are 60 misses in the first-level cache, 30 misses in the second-level cache, and 5 misses in the third-level cache. Assume the miss penalty from the L3 cache to memory is 100 clock cycles, the hit time of the L3 cache is 10 clocks, the hit time of the L2 cache is 5 clocks, the hit time of L2 is 1 clock cycle, and there are 1.5 memory references per instruction.

1. What's the global miss rate for each level of caches?
2. What's the local miss rate for each level of caches?
3. What is the average memory access time?
4. What is the average stall cycle per instruction?

1. $L1 = 60/1000 = 0.06$, $L2 = 30/1000 = 0.03$, $L3 = 5/1000 = 0.005$
2. $L1 = 60/1000 = 0.06$, $L2 = 30/60 = 0.5$, $L3 = 5/30 = 0.167$
3. $AMAT = 1 + 0.06 * 5 + 0.03 * 10 + 0.005 * 100 = 2.1 \text{ clock cycles}$
4. $(2.1 - 1) * 1.5 = 1.65 \text{ clock cycles}$

練習 : Assume that main memory accesses take 70ns and that memory accesses are 36% of all instructions. The following table shows data for L1 caches attached to each of two processors, P1 and P2.

	L1 size	L1 miss rate	L1 hit time
P1	1KB	11.4%	0.62ns
P2	2KB	8.0%	0.66ns

1. Assuming that the L1 hit time determines the cycle times for P1 and P2, what are their respective clock rates?
2. What is the AMAT for each of P1 and P2?
3. Assuming a base CPI of 1.0, what is the total CPI for each of P1 and P2? Which processor is faster?

For the next three problems, we will consider the addition of an L2 cache to P1 to presumably make up for its limited L1 cache capacity. Use the L1 cache capacities and hit times from the previous table when solving the following problems. The L2 miss rate indicated is its local miss rate.

L2 size	L2 miss rate	L2 hit time
512KB	98%	3.22ns

4. What is the AMAT for P1 with the addition of an L2 cache? Is the AMAT better or worse with the L2 cache?
5. Assuming a base CPI of 1.0, what is the total CPI for P1 with the addition of an L2 cache?
6. Which processor is faster, now that P1 has an L2 cache? If P1 is faster, what miss rate would P2 need in its L1 cache to match P1's performance? If P2 is faster, what miss rate would P1 need in its L1 cache to match P2's performance?

1	2			3	
P1	1.61GHz	8.60ns	13.87 cycles	18.5	P2
P2	1.52GHz	6.26ns	9.48 cycles	12.54	

4			5
8.81ns	14.21 cycles	Worse	18.96
6	P1 with L2 cache: CPI = 18.96. P2: CPI = 12.54 P2 is still faster than P1 even with an L2 cache. The miss rate for P1 in its L1 cache should be 7.83% to match P2's performance.		

練習：(前略) AMAT is useful as figure of merit for different cache systems.

- Find the AMAT for a processor with a 2ns clock, a miss penalty of 20 clock cycles, a miss rate of 0.05 misses per reference, and a cache access time (including hit detection) of 1 clock cycle. Assume that the read and write miss penalties are the same and ignore other write stalls.
- Suppose we can improve the miss rate to 0.03 misses per reference by doubling the cache size. This causes the cache access time to increase to 1.2 clock cycles. Using the AMAT as a metric, determine if this is a good trade-off.
- If the cache access time determines the processor's clock cycle time, which is often the case, AMAT may not correctly indicate whether one cache organization is better than another. If the processor's clock cycle time must be changed to match that of a cache, is this a good trade-off? Assume the processors are identical except for the clock rate and the number of cache miss cycles; assume 1.5 references per instruction and a CPI without cache misses of 2. The miss penalty is 20 cycles for both processors.

- $AMAT = 2ns + 0.05 * (20 * 2ns) = 4ns$
- $AMAT = (1.2 * 2ns) + (20 * 2ns * 0.03) = 2.4ns + 1.2ns = 3.6ns$
Yes, it's a good choice.
- $Execution\ time_{old} = 2 * IC * (2 + 1.5 * 20 * 0.05) = 7IC$
 $Execution\ time_{new} = 2.4 * IC * (2 + 1.5 * 20 * 0.03) = 6.96IC$
So, it's a good choice.

例(82)：

- Average memory access time (AMAT) is the average time to access memory considering both hits and misses and the frequency of different access. It is defined as:
 $AMAT = \text{Time for a hit} + \text{Miss rate} * \text{Miss penalty}$
If the machine A with a 10-ns clock, a miss penalty of 20 clock cycles, a miss rate of 0.05 misses per reference, and a cache access time including hit detection of 1 clock cycle. Assume the read and write miss penalties are the same and ignore other write stalls. If the machine B has the miss rate 0.03 misses per reference by doubling the cache size. This causes the cache access time to increase to 1.2 clock cycles. Using the AMAT as a metric to determine which machine is better.
- If the machine's clock cycle time must be changed to match that of a cache, which machine is better based on the measurement of execution time. Assume the machines are identical except for the clock rate and number of cache miss cycles; assume 1.5 references per instruction and a CPI without cache misses of 2. The miss penalty is 20 cycles for both machines.

- $AMAT\ for\ machine\ A = (1 + 0.05 * 20) * 10ns = 20ns$
 $AMAT\ for\ machine\ B = (1.2 + 0.03 * 20) * 10ns = 18ns$
So, machine B is better.
- $Execution\ Time = \text{Clock cycle} * IC * (CPI_{base} + \text{Cache miss cycles per instruction})$
 $Execution\ time\ for\ machine\ A = 10 * IC * (2 + 1.5 * 20 * 0.05) = 35 IC$
 $Execution\ time\ for\ machine\ B = 12 * IC * (2 + 1.5 * 20 * 0.03) = 34.8IC$
Hence, machine B is better

架構

- (1) Motivation
- (2) Mapping : Memory $\Leftrightarrow$ Hard Disk
- (3) Ignore cache temporary

- (1) Mapping : Memory $\Leftrightarrow$ Virtual Space
- (2) CPU $\rightarrow$ Virtual address

- (1) Block $\Leftrightarrow$ Page
Miss $\Leftrightarrow$ Page fault
- (2) High miss penalty results in
 - a. Mapping
 - b. Write
 - c. Replacement
 - d. Block size

- (1) Page Table
- (2) PTR
- (3) Memory Management bits
- (4) Page Table size

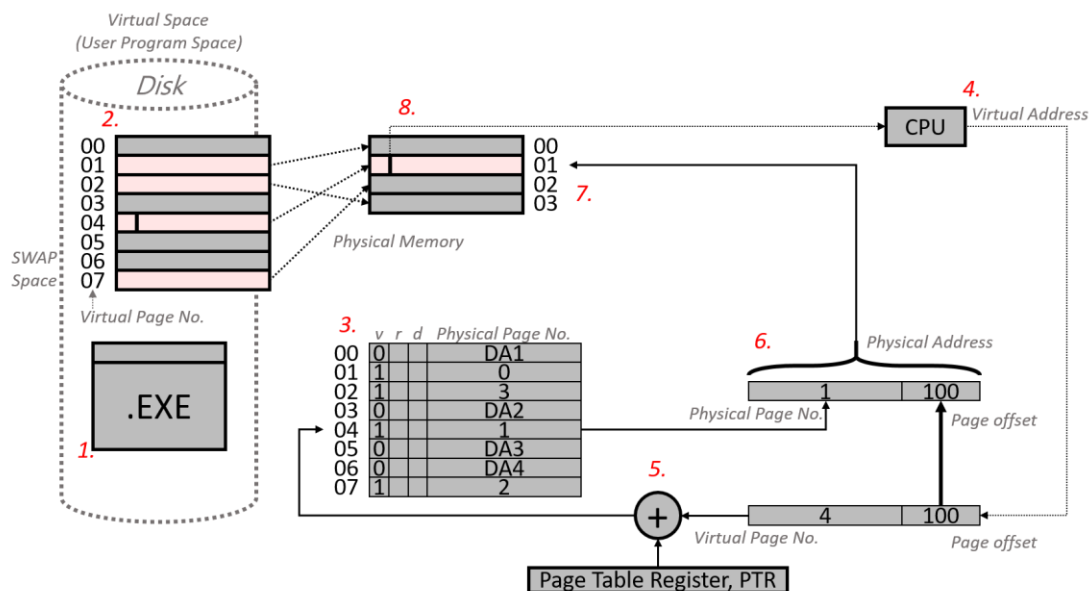
1. 允許多個程式能夠有效率且安全的分享記憶體
2. 消除因主記憶體過小所造成程式的限制

1. 允許多個程式能夠有效率且安全的分享記憶體
2. 消除因主記憶體過小所造成程式的限制

	Terminologies			Strategies				Search
				Mapping	Write	Replace	Block	
Cache	Block (Line)	Memory Block Address	Miss	Fully Set Direct	Write Through Write Back	Random LRU	Medium	Index to block or set them compare tag
		Cache Block Address						
Virtual Memory	Page	Virtual Page Number	Page Fault	Fully (有最小的分頁 失誤率)	Write Back (因為硬碟速 度實在太慢)	LRU	Large (4KB~ 16KB)	Lookup page table
		Physical Page Number						

執行程式與取得資料的過程：

1. 開起執行檔(滑鼠點擊...等方式)
2. OS 把程式複製到硬碟 SWAP Space 之中，複製後，程式本身稱為 Virtual Space(又稱為 User Program Space)
3. 當程式使用到的空間較記憶體還要大時，OS 會先把部分資料複製到 Physical Memory，並建立 Page Table，以 Physical Page Number 來記錄已經載入(數值)、與還在硬碟內未載入(硬碟位置)的資料內容。Page Table 會與程式本身資料量相當(圖示皆為 8 個)，並會另外增加 Memory Management bits(Valid bit、Reference bit、Dirty bit，後面會再介紹到)
4. 當 CPU 要存取某一資料時，會發出 Virtual Address 的要求指令(直接指定硬碟內位置)，Virtual Address 是由兩部分組成：Virtual Page Number 與 Page offset，前者是對應到硬碟中程式本身的區塊、後者為區塊內的位置
5. 因為 CPU 計算的資料來源為記憶體，故 Virtual Page Number 會需要轉換為 Memory 中的資料，因此會先透過加上 Page Table Register 的數值，以找到 Page Table，再從 Page Table 取得對應的 Physical Page Number(圖中即是先將第 4 個 Virtual Page Number 加上 PTR 數值，到 Page Table 第 4 號，確認 Valid bit 為 1，再找到對應 Physical Page Number 為 1)
6. 有了 Physical Page Number，再結合先前的 Page Offset，便組合成了 Physical Address
7. 在 Memory 中，以對應的 Physical Address 取得資料(在 Memory 第 1 區塊內第 100 的位置)
8. 回傳給 CPU 進行運算，完成資料取得



例(47) : Which of the following must be saved during a context switch so that process may resume?

1. All general purpose register
2. PC (Program Counter)
3. All pipelined registers
4. The Page Table Register

1、2、4

Memory Management Bits

1. **Valid bit** 有效位元：表示所需要存取的資料是否在記憶體中，且若為 0 則代表『分頁錯誤』的發生(Cache 中的 Miss)
2. **Reference bit** 參考位元(又稱 *Use Bit* 使用位元)：完整實作 LRU 的成本過高，因此大多是以『近似 LRU』的方式完成，而 **Reference bit** 即紀錄了使用次數，可用來選出要被置換出去的 **Victim Page** 犧牲分頁
3. **Dirty bit** 修改位元：當有資料改寫入 **Memory** 中時，因為硬碟速度過慢，不會即時被更新，因此 **Dirty bit** 就會設定成 1，以提示之後 **Write Back** 時要更新硬碟內資料

例(90) : To track whether a page has been written since It was read into the memory, a __ is added to the page table.

1. valid bit
2. tag index
3. dirty bit
4. reference bit

3

例(89) : __ implements the translation of a program's address space to physical addresses.

1. DRAM
2. Main memory
3. Physical memory
4. Virtual memory

4

例(123) : Consider a virtual memory system with the following properties:

1. 40-bit virtual address
2. 16 KB pages
3. 36-bit physical address

Assume that the valid, protection, dirty, and use bits take a total of 4 bits and that all the virtual pages are in use. Assume that disk addresses are not stored in the page table. What is the total size of the page table for each process on this machine?

$$\text{Number of page table entries} = 2^{40}/2^{14} = 2^{26}$$

$$\text{The size of physical page No.} = 36-14 = 22$$

$$\text{The size of each entry} = 22+4 = 26 \text{ bits.}$$

$$\text{Total size of the page table} = [26/32] * 2^{26} \text{ word} = 256 \text{ MB}$$

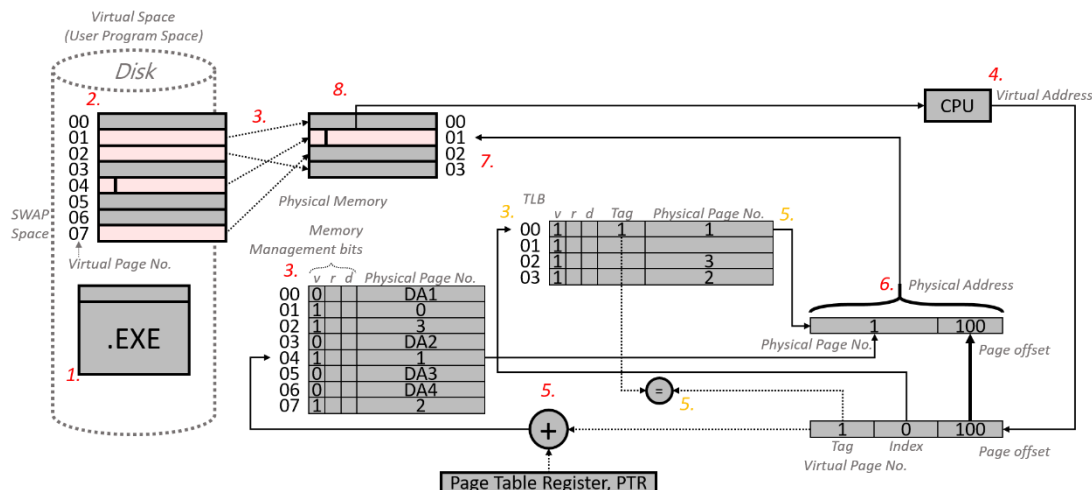
重點九：加速位址轉換(TLB)

緣由：Page 的方式需要存取 2 次 Memory(一次是 Page Table 獲得實體位址、一次是取得資料)，故若能改善、只存取 1 次，可提升速度。故利用 Cache 概念，使用一個特別的快取來記錄最近使用過的轉換資訊，稱為 TLB(Translation-Lookaside Buffer)

TLB 的主要用處是加速虛擬位址轉為實體位址。方式是新增一個類似 Cache 的暫存器，當 Virtual Page Number 要去生成 Physical Page Number 的同時，直接比較 TLB 內、近期的轉換記錄，若是存在，則直接轉出 Physical Page Number 去合成 Physical Address，轉換速度會極大地提升。

而方式如同 Cache，但亦可靠軟體來實作(Page Table 轉換時的第 3 至 5 步驟)：

1. 切出 Index，至 TLB 相同 Index 位置
2. 比較 Tag 是否相同
3. 若是相同則表示已找到，直接轉傳出 Physical Page Number



例(11)：

1. What is TLB, please spell out and explain it?
2. Why we need TLB in virtual memory?

1. TLB (Translation-Lookaside Buffer): a cache that keeps track of recently used address mappings to avoid an access to the page table.
2. To speedup the translation from virtual address to physical address.

例(88)：A virtual memory system often implements a TLB to speed up the virtual-to-physical address translation. A TLB has the following characteristics. Assume each TLB entry has a valid bit, a dirty bit, a tag, and the page number. Determine the exact total number of bits to implement this TLB.

- It is direct-mapped
- It has 16 entries
- The page size is 4Kbytes
- The virtual address space is 4Gbytes
- The physical memory is 1 Gbytes

The length of virtual page number = $\log_2(4 \text{ Gbytes}/4\text{K bytes}) = 20 \text{ bits}$

The length of physical page number = $\log_2(1 \text{ Gbytes}/4\text{K bytes}) = 18 \text{ bits}$

The index field = $\log_2 16 = 4 \text{ bits}$

The tag field = $20 - 4 = 16 \text{ bits}$

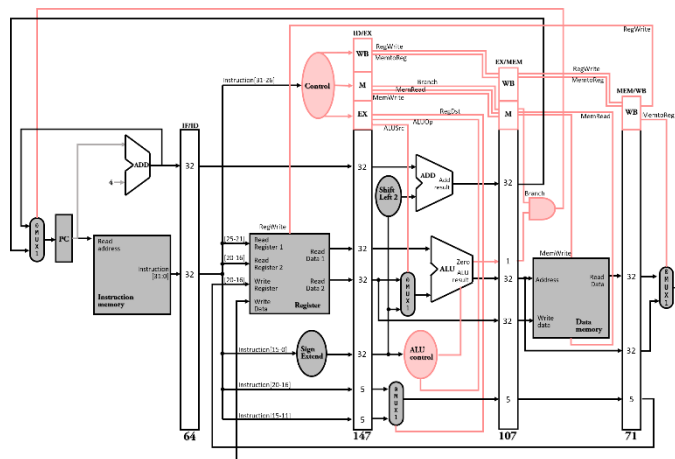
The bits in each entry of TLB = $2 + 16 + 18 = 36$

The size of TLB = $36 * 16 = 576 \text{ bits}$

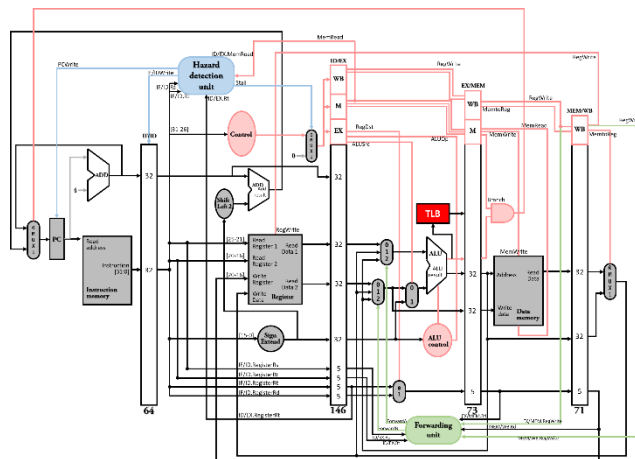
例(14) : Design a memory hierarchy system for a conventional five-stage pipeline processor. Assume both the virtual address and physical address are 32 bits. The page size is 32 KB. ITLB has 48 entries and 3-way set associative. DTLB is direct-mapped, 32 entries. The data cache is physically addressed; cache size 32KB, direct-mapped, line size 64 bytes.

1. Show a typical five stage pipeline diagram and explain their functionality.
2. Show how to integrate the two TLBs to the pipeline.
3. Explain the translation of virtual address to the physical address for ITLB.
4. Design the data cache and integrate it with the DTLB by showing how the address lines are used.

1.



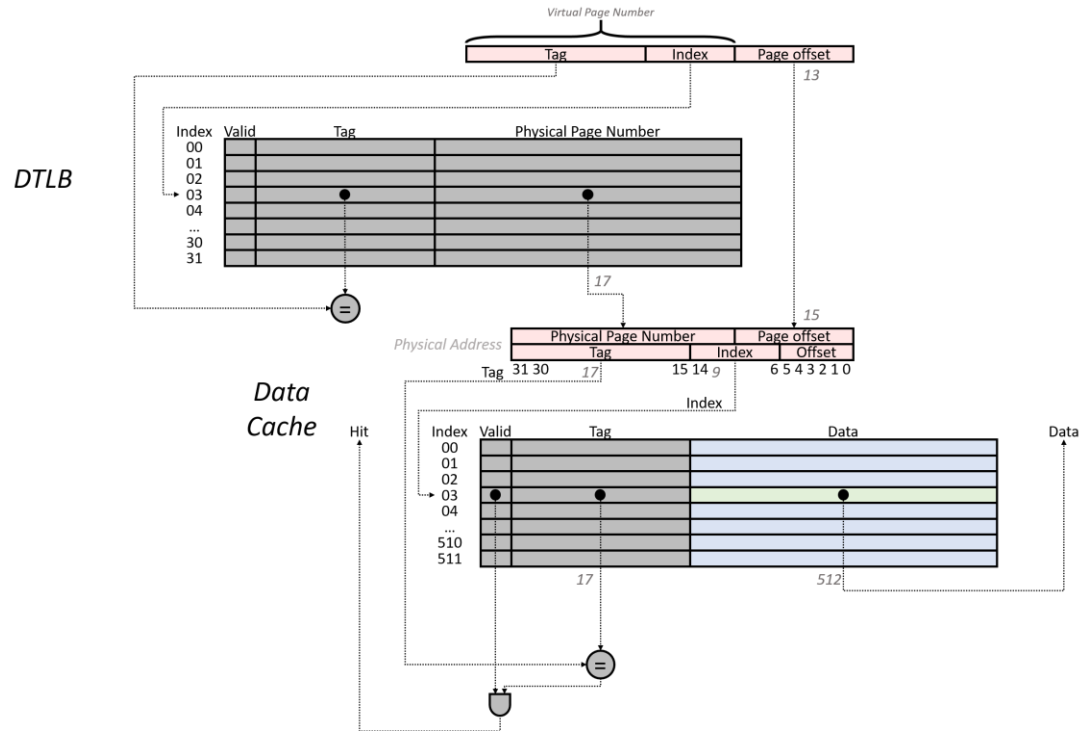
2.



之所以設置在 EXE 階段，是不希望 MEM 階段時間過長

3. The virtual address is first broken into a virtual page number and a page offset. The ITLB which contains the virtual to physical address translation is indexed by the virtual page number. If the mapping exists, the physical page number mapped from the ITLB constitutes the upper portion of the physical address, while the page offset constitutes the lower portion. If the mapping does not exist, the missing translation will be retrieved from the page table.

4.



例(36) : Which of the following is (are) true?

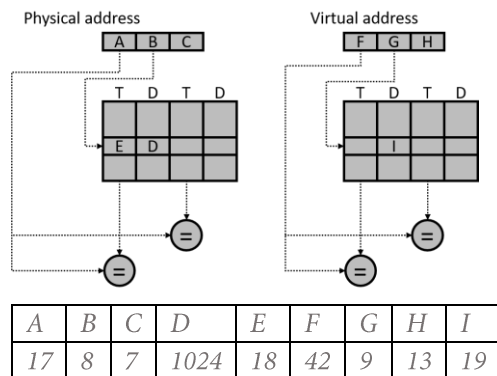
1. A page fault is signaled by hardware.
2. A TLB exception can also result in a page fault
3. A cache miss is typically handled by software.
4. A page fault is typically handled by hardware.

1

練習 : Consider a memory system with the following parameters:

- Translation Lookaside Buffer has 512 entries and is 2-way set associative.
- 64Kbyte L1 Data Cache has 128 byte lines and is also 2-way set associative.
- Virtual addresses are 64-bits and physical addresses are 32 bits.
- 8KB page size.

Below are diagrams of the cache and TLB. What is the number of bits in the diagrams labeled A, B, ...



例(53) : 同上題

練習：The following table is a stream of virtual addresses. Assume 4KB pages, a four-entry fully associative TLB, and true LRU replacement. If pages must be brought in from disk, increment the next largest page number.

4095, 31272, 15789, 15000, 7193, 4096, 8912

Valid	Tag	Physical Page Number
1	11	12
1	7	4
1	3	6
0	4	9

Page table

Valid	Physical page or in disk	Valid	Physical page or in disk
1	5	0	Disk
0	Disk	1	4
0	Disk	0	Disk
1	6	0	Disk
1	9	1	3
1	11	1	12

Given the address stream above, and the shown initial state of the TLB and page table, show the final state of the system. Also list for each reference if it is a hit in the TLB, a hit in the page table, or a page fault.

(H: Hit in TLB : M: Miss in TLB hit in page table : PF: Page Fault)

VA	4095	31272	15789	15000	7193	4096	8912
VPN	0	7	3	3	1	1	2
H/M/PF	M	H	H	H	PF	H	PF

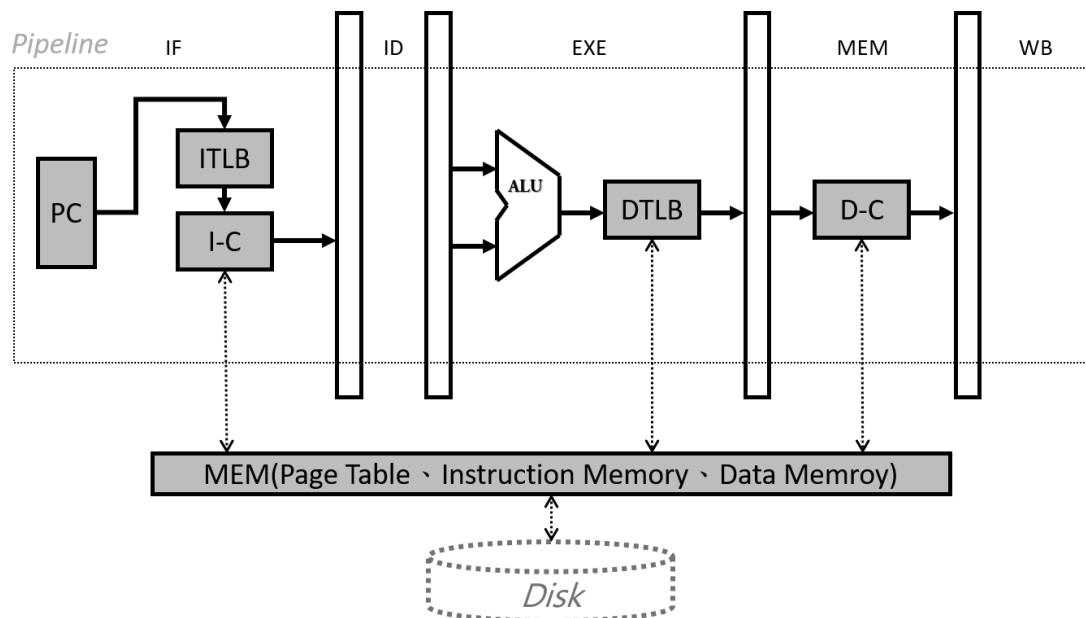
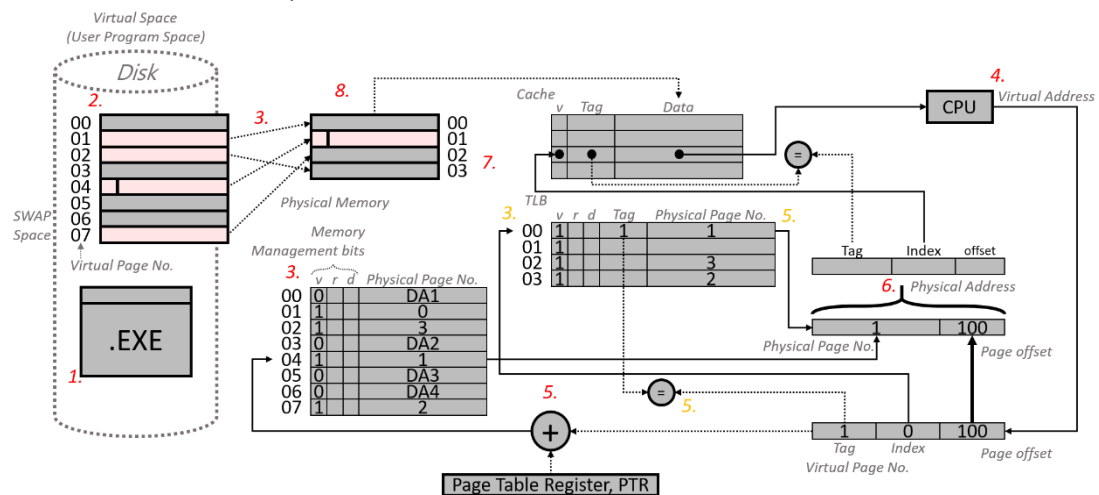
TLB

Valid	Tag	Physical Page Number
1	1	13
1	7	4
1	3	6
1	2	14

Page table

Valid	Physical page or in disk	Valid	Physical page or in disk
1	5	0	Disk
1	13	1	4
1	14	0	Disk
1	6	0	Disk
1	9	1	3
1	11	1	12

CPU、Virtual Memory、TLB、Cache 整合圖：



例(80)：Assume that a processor implements the hardware TLB cache. Assume also that instruction and data caches are physical address caches. The processor includes the write buffer into its implementation.

1. For executing an instruction, what is the maximal number of cache misses that can be introduced to the processor? (need not consider the misses due to the exceptions handling)
2. What are these misses?
3. What is the function of the write buffer?

1. 4
2. TLB miss, instruction cache miss, TLB miss, and data cache miss.
3. The function of the write buffer is to store the data while it is waiting to be written to memory. After writing the data into the cache and into the write buffer, the processor can continue execution.

TLB	Page table	Cache	Possible? If so, under what circumstance?
H	H	M	Possible, although the page table is never really checked if TLB hits.
M	H	H	TLB misses, but entry found in page table: after retry, data is found in cache.
M	H	M	TLB misses, but entry found in page table: after retry, data misses in cache.
M	M	M	TLB misses, and is followed by a page fault; after retry, data must miss in cache.
H	M	M	Impossible: cannot have a translation in TLB if page is not present in memory.
H	M	H	Impossible: cannot have a translation in TLB if page is not present in memory.
M	M	H	Impossible: data cannot be allowed in cache if the page is not in memory.

例(42)：In a memory hierarchy with a TLB and a cache, a memory reference can encounter three different types of misses: a TLB miss, a page fault, and a cache miss. Consider all the combinations of these three event with one or more occurring (seven possibilities). For each possibility, state whether this even can actually occur and under what circumstances.

TLB	Page table	Cache	Possible? If so, under what circumstance?
H	H	M	Possible, although the page table is never really checked if TLB hits.
M	H	H	TLB misses, but entry found in page table: after retry, data is found in cache.
M	H	M	TLB misses, but entry found in page table: after retry, data misses in cache.
M	M	M	TLB misses, and is followed by a page fault; after retry, data must miss in cache.
H	M	M	Impossible: cannot have a translation in TLB if page is not present in memory.
H	M	H	Impossible: cannot have a translation in TLB if page is not present in memory.
M	M	H	Impossible: data cannot be allowed in cache if the page is not in memory.

例(122)：In a computer memory hierarchy, determine which of the following five combinations of events (for locating a page of memory) in the cache, TLB, page table, and main memory are possible to occur. Answer Yes or No to each of 1~5.

	TLB	Page Table	Cache	Main Memory
1	H	H	H	M
2	H	H	M	M
3	M	H	H	M
4	M	M	H	H
5	H	M	H	H

1.No 、2.No 、3.No 、4.No 、5.No

練習：將左邊的記憶體階層的元素配對到右邊最接近的說明：

1. L1 快取	a. 快取的快取
2. L2 快取	b. 磁碟的快取
3. 主記憶體	c. 主記憶體的快取
4. TLB	d. 分頁表項目的快取

1-a 、2-c 、3-b 、4-d

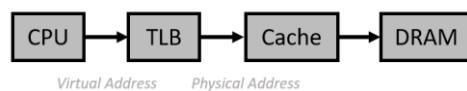
練習：TLB's are typically built to be fully-associative or highly set-associative. In contrast, first-level data caches are more likely to be direct-mapped or 2 or 4-way set associative. Give two good reasons why this is so.

1. 對第一層快取而言，使用直接或低關聯度對映方式具有降低 CPU 存取時間的優點。
2. 對 TLB 而言，完全關聯式或高關聯度的對映方式有較小的失誤率，此外 TLB 較小，所以完全關聯式或高關聯度對映方式的成本不會太高

重點十：虛擬位址快取

我們之前都是假設：先將所有的位址轉成實體位址後、才開始動作。但這種架構下，快取是使用實體索引(Physically Indexed)與實體標籤(Physically Tag)，若是命中的話，存取記憶體將花費兩倍的時間(TLB 存取、與快取存取)，效能較差

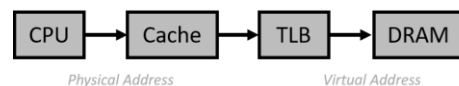
Physical Addressed Cache



1. Translate first from virtual address to physical address
2. Access with physical address

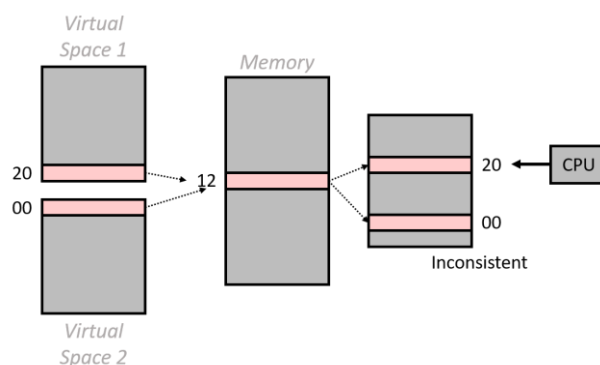
然而，CPU 也可以全部、或部分使用虛擬位址(虛擬索引與虛擬標籤)來進行，稱之為『虛擬位址快取 Virtually Address Cache』。如此一來，只有當 miss，需要存取 Memory 時，才轉換 Virtual Address 成 Physical Address：

Virtual Addressed Cache



1. Access Cache first
2. Only translate if going to main memory

虛擬位址快取的缺點是：可能會有一個分頁(實體位址 12 號)，但被兩個程式共享資料(虛擬位址 00 與 20 號)，但 Memory 內只會有一組(實體位址 12 號)、但 Cache 內會有兩組。故當其中一個程式改變資料時(虛擬位址 20 號)，另一個程式對應的(虛擬位址 00 號)資料卻不知道要跟著改變，稱之為『別名 Aliasing』，因此需要有適當的解決方式：『虛擬索引(Virtually Indexed)、實體標籤(Physically Tagged)』



虛擬索引(Virtually Indexed)、實體標籤(Physically Tagged)：
CPU 要求時，先將 Virtual Address 切出 Index，到 Cache 中找到對應的區塊；同時透過 TLB 轉出 Physical Address，切出 Tag，並與 Cache 中、該 Index 區塊中的(Physical) Tag 做比較，最後將 Data 回傳給 CPU。此種方式因為 Physical Tag 是統一的，故不會有 Aliasing 狀況發生，不過需要配合軟體協助

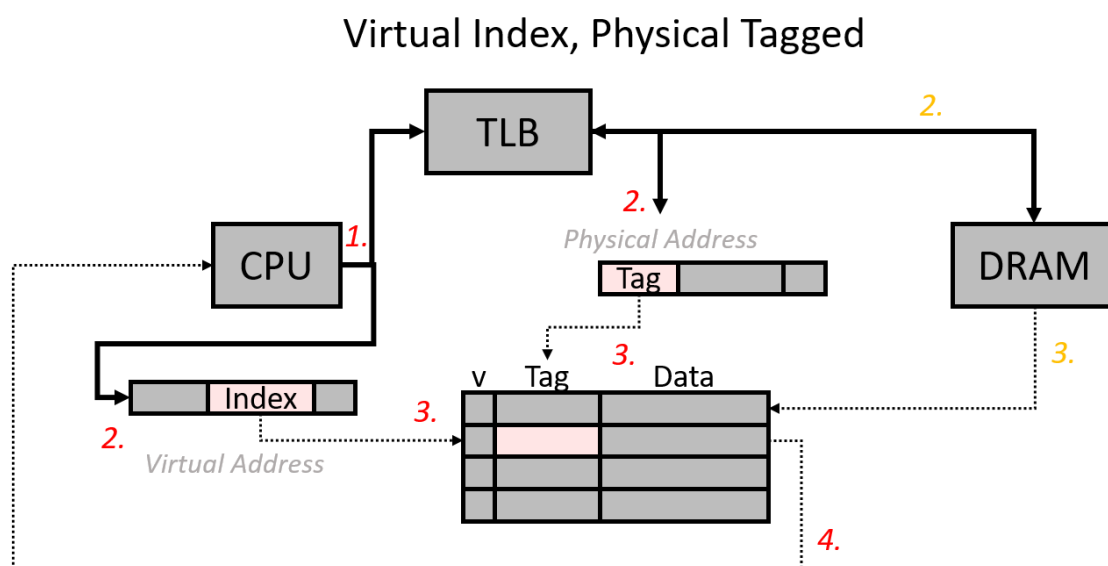
過程如下(課本已刪)：

Hit 路線(紅色)：

1. CPU 提出 Virtual Address
2. 同步執行：(1)Virtual Address 切出 Index；(2)TLB 轉成 Physical Address，並切出 Tag
3. 同步執行：(2)至 Cache 找到對應的 Entry；(2)比較對應 Entry 的 Tag
4. 若是相同，則回傳 Data 給 CPU

Miss 路線(橘色)：

1. CPU 提出 Virtual Address
2. TLB Miss 時，至 Memory(DRAM)找出正確資料
3. 將正確資料同步 Cache
4. 回傳 Data 給 CPU



優缺點：

	Virtually Indexed, Virtually Tagged	Virtually Indexed, Physically Tagged	Physically Indexed, Physically Tagged
優點	效率最佳	效率次佳、架構簡單	架構簡單
缺點	會有別名問題	總是要轉換虛擬位址至實體位址	效率最差

練習：(前略)Describe the virtues and drawbacks for each of these systems.

	<i>Virtually Indexed, Virtually Tagged</i>	<i>Virtually Indexed, Physically Tagged</i>	<i>Physically Indexed, Physically Tagged</i>
優點	效率最佳	效率次佳、架構簡單	架構簡單
缺點	會有別名問題	總是要轉換虛擬位址至實體位址	效率最差

例(69)：Virtually addressed cache is accessed with a virtual address, while physically addressed cache is accessed with a physical address.

- Which kind of cache takes the TLB out of normal cache access path, reducing cache latency?
- Which kind of cache has aliasing problem?
- Assume physically addressed cache is used, is it possible to have TLB miss, but page table and cache hit for a data access?
- Assume virtually addressed cache is used, is it possible to have TLB miss, but page table and cache hit for a data access? Please explain your answer

- Virtually addressed cache*
- Virtually addressed cache*
- Yes. TLB miss, but entry found in page table; after retry, data is found in cache.*
- Yes. When data is in both cache and memory and mapping in page table but not in TLB*

例(71)：Fill in the best appropriate answer for each of the following questions.

- When the same object is accessed by two addresses, for instance, two virtual addresses for the same physical page in virtual memory, we call this situation ____.
- A ____ is a changing of the internal state of processor to allow a different process to use the processor.
- A cache miss which is caused by the first access to a block that has never been in the cache is called a ____ miss.
- The virtual memory treats the main memory as a fully set associative cache which uses ____ policy for handing write hits.

- Aliasing*
- Context switch*
- Compulsory*
- Write back*

例(91)：Assume all memory addresses are translated to physical addresses before the cache is accessed. In this case, the cache is physically indexed and physically tagged. Also assume a TLB is used.

- Under what circumstance can a memory reference encounter a TLB miss, a page table hit, and a cache miss?
- To speed up cache accesses, a processor may index the cache with virtual addresses. This is called a virtually addressed cache, and it uses tags that are virtual addresses. However, a problem called aliasing may occur. Explain what aliasing is and why.
- In today's computer systems, virtual memory and cache work together as a hierarchy. When the operating system decides to move a page back to disk, the contents of that page may have been brought into the cache already. What should the OS do with the contents that are in the cache?

- Data/Instruction is in memory but not in cache and page table has this mapping but TLB has not.*
- A situation in which the same object is accessed by two addresses; can occur in virtual memory when there are two virtual addresses for the same physical page.*

3. If the contents in cache are dirty, force them write back to memory and invalidate them in cache. After that, copy the page back to disk. If not, invalidate them in cache and copy the page back to disk.

例(78) : For the following memory system parameters:

-Cache: virtually addressed but physically tagged, 8KB, direct mapped, 32 B block size

-TLB: fully associative, 32 entries

-32 bit physical address

-Virtual Memory: 8KB page, 64 bit virtual address

1. How many bits are needed in each tag, index, and block offset fields of the cache?
2. How many bits are needed in each tag field of the TLB?
3. Why do we want to use a virtually addressed but physically tagged cache?

1. The number of blocks in the cache = $8KB/32B = 256 = 2^8$

The number of bits in the index field = 8

The number of bits in the tag field = $32-8-5 = 19$

The number of bits in the block offset field = $5-2 = 3$

Tag	Index	Block offset	Byte offset
19	8	3	2

2. The number of bits in the tag field of the TLB = $64-13 = 51$

3. When the cache is accessed with a virtual address and pages are shared between programs, there is the possibility of aliasing. A virtually addressed but physically tagged cache can avoid aliasing.

重點十一：虛擬記憶體的保護機制

與共享記憶體雷同，硬體需支持：

1. 兩種模式 Dual Mode：User mode 與 Kernel mode
2. Mode bit、Page Table Pointer、TLB
3. 例外返回 (Return from exception, ERET)，以回到 User mode，並跳回 EPC 位址

例(94)：

1. Why can the virtual memory mechanism provide the memory protection among processes in a multi-processing environment?
2. Describe how a virtual address is translated into a physical address.
3. What is the translation lookaside buffer (TLB) designed for? How does it work?
4. What are the benefits if a larger page size is chosen? (List at least 3.) What is the draw back? (List at least 1)

1. The hardware provides three basic capabilities to protect among processes.

(1) Support two modes that indicate whether the running process is a user process or an operating system process called supervisor process.

(2) Provide a portion of the processor state that a user process can read but not write.

(3) Provide mechanisms whereby the processor can go from user mode to supervisor mode, and vice versa.

2. In virtual memory systems, page table is used to translate a virtual address into a physical address.

The page table containing the virtual to physical address translations.

3. TLB is used to make address translation fast. TLB (a cache of a page table) keep track of recently used address mappings to avoid an access to the page table.

- 4.

Benefits	1. <i>More efficient to amortize the high access time</i> 2. <i>More spatial and temporal localities</i> 3. <i>Smaller page table size</i>
Drawbacks:	1. <i>More internal fragmentation</i> 2. <i>Higher miss penalty</i>

重點十二：記憶體階層的 4 個問題

問題 1：區塊可被放在哪個地方？

回答 1：只有一個地方(直接對映)、一些地方(集合關聯式)、或任何地方(完全關聯式)

Scheme name	Number of sets	Blocks per set
Direct mapped	Number of blocks in cache	1
Set associative	Number of blocks in cache / Associativity	Associativity (typically 2-16)
Fully associative	1	Number of blocks in the cache

-增加關聯度的優點是會降低失誤率，而失誤率改善來自於降低了因為競爭同一空間而造成的失誤

-當快取增加，從關聯度增加獲得的改進變小，因此較大快取的整體失誤率較小，所以可改進的空間也較少，因此關聯度所改善的失誤率也明顯減少

問題 2：如何找到一個區塊？

回答 2：有 4 種方式：Index(Direct mapped)、有限搜尋(Set associative)、全部搜尋(Fully associative)、與分離的對應表(Page Table)

問題 3：快取失誤時，該置換哪個區塊？

回答 3：方式有二：Random 與 LRU

問題 4：寫入時會發生什麼事？

回答 5：Data Inconsistent；解決方式有二：Write Through 與 Write Back

例(114)：

1. Explain the differences between a write-through policy and a write back policy.
2. Tell which policy cannot be used in a virtual memory system, and describe the reason.

1. *Write through: always write the data into both the cache and the memory.*
Write back: updating values only to the block in the cache, then writing the modified block to the lower level of the hierarchy when the block is replaced.
2. *Write through will not work for virtual memory, since writes take too long. Instead, virtual memory systems use write back.*

重點十三：3C(三種記憶體階層失誤來源的分類)

1. 強迫性失誤(Compulsory Misses)：若一區塊從未被使用過，則第一次讀取時，此區塊必定不會在快取中，即發生強迫性失誤。也稱為冷開機失誤(Cold-Start Misses)
2. 容量性失誤(Capacity Misses)：當快取的容量不足以包含一程式執行所需之區塊時，就會發生快取的容量性失誤。因為區塊會不斷被取代、替換，而需要重新載入
3. 衝突性失誤(Conflict Misses)、也稱碰撞失誤(Collision Misses)：只發生在關聯式、與直接映射快取；當許多區塊映射到同一區塊或集合時，所發生的快取失誤。此種失誤在相同容量的『完全關聯式快取』中，不會發生。

	Compulsory	Capacity	Conflict
Fully	O	O	
Set	O		O
Direct	O		O

練習：以下的哪些敘述(如果有的話)通常是對的？

1. 沒有辦法降低強迫性失誤
2. 完全關聯式快取沒有衝突失誤
3. 對於降低失誤次數來說，關聯度比容量重要的多

2.

註(1)：增加 Block size；註(3)：容量比較重要

記憶體階層設計上的挑戰

改善的設計	Miss Rate 的影響	可能的負面影響
增加 Block size	1. 降低強迫性失誤 2. 因為空間區域性，Miss Rate 會降低 (不過若 Block size 太大，Miss Rate 又會再增加)	Miss Penalty 會增加
增加 Cache size	減少容量性失誤	可能增加存取時間
增加關聯度	減少由衝突性失誤所造成的 Miss Rate	可能增加存取時間

例(33)：The following table summarizes the memory hierarchy parameters and their effects on performance. Copy this table to your exam paper and fill in each missing cell with the most proper statement from below.

Design Change	Potential advantages	Possible Disadvantages
Increase cache size		
Increase block size		
Increase associativity		
More sophisticated replacement policy		

- | | | |
|-------------------------------|-----------------------------|-------------------------------|
| 1. Decrease capacity misses | 5. Decrease decision time | 9. Increase compulsory misses |
| 2. Decrease conflict misses | 6. Decrease miss penalty | 10. Increase access time |
| 3. Decrease compulsory misses | 7. Increase capacity misses | 11. Increase decision time |
| 4. Decrease access time | 8. Increase conflict misses | 12. Increase miss penalty |

<i>Design Change</i>	<i>Potential advantages</i>	<i>Possible Disadvantages</i>
<i>Increase block size</i>	<i>3.Decrease Compulsory Misses</i>	<i>12.Increase Miss Penalty</i>
<i>Increase cache size</i>	<i>1.Decrease Capacity Misses</i>	<i>10.Increase Access Time</i>
<i>Increase associativity</i>	<i>2.Decrease Conflict Misses</i>	<i>10.Increase Access Time</i>
<i>More sophisticated replacement policy</i>	<i>2.Decrease Conflict Misses</i>	<i>11.Increase Decision Time</i>

例(56) : Which of the following can reduce Cold (or Compulsory) misses?

1. Larger cache line size
2. Sequential cache prefetching(預先置入)
3. Higher associativity
4. LRU replacement algorithm

1、2

例(46) :

1. What is the 3Cs model for cache misses?
2. What design changes can be used to improve any one of the 3Cs?
3. What possible negative effect may occur for each the design changes?

1. *3Cs model: a cache model in which all cache misses are classified into one of 3 categories: compulsory, capacity, and conflict misses.*

	<i>Compulsory</i>	<i>Capacity</i>	<i>Conflict</i>
2.	<i>Increase block size</i>	<i>Increase cache size</i>	<i>Increase associativity</i>
3.	<i>Increase miss penalty</i>	<i>Increase access time</i>	<i>Increase access time</i>

3C 實戰題

策略：

	Compulsory	Capacity	Conflict
Fully	O	O	
Set	O		O
Direct	O		O

1. 先判斷是哪一種 Associativity
2. 再看是否為 Compulsory，若是則 Case Close；若否則為另一種

例(92) : Assume the three caches below, each consisting of 16 words. Given the series of address references as word addresses: 2, 3, 4, 16, 18, 16, 4, 2. Please label each reference as a hit or a miss for the three caches 1、2、3 below. Assuming that LRU is used for cache replacement algorithm and all the caches are initially empty.

1. A direct-mapped cache with 16 one-word blocks
2. A direct-mapped cache with 4 four-word blocks
3. A four-way set associative cache with block size of one-word.

1. 表

Word address		Tag	Index	Hit/Miss	3C
Decimal	Binary				
2	00010	0	0010	Miss	Compulsory
3	00011	0	0011	Miss	Compulsory
4	00100	0	0100	Miss	Compulsory
16	10000	1	0000	Miss	Compulsory
18	10010	1	0010	Miss	Compulsory
16	10000	1	0000	Hit	
4	00100	0	0100	Hit	
2	00010	0	0010	Miss	Conflict

2. 表

Word address		Tag	Index	Hit/Miss	3C
Decimal	Binary				
2	00010	0	00	Miss	Compulsory
3	00011	0	00	Hit	
4	00100	0	01	Miss	Compulsory
16	10000	1	00	Miss	Compulsory
18	10010	1	00	Hit	
16	10000	1	00	Hit	
4	00100	0	01	Hit	
2	00010	0	00	Miss	Conflict

3. 表

Word address		Tag	Index	Hit/Miss	3C
Decimal	Binary				
2	00010	000	10	Miss	Compulsory
3	00011	000	11	Miss	Compulsory
4	00100	001	00	Miss	Compulsory
16	10000	100	00	Miss	Compulsory
18	10010	100	10	Miss	Compulsory
16	10000	100	00	Hit	
4	00100	001	00	Hit	
2	00010	000	10	Hit	

例(66) : Consider a computer has a fully associate mapped 32 bytes cache with block size of 8 bytes. The replacement policy is LRU. A program is run on this machine. The memory access sequence in word address is 0, 1, 2, 5, 6, 7, 3, 1, 2, 3, 4, 5, 6, 7, 20, 21, 22, 23, 0, 1. What is the cache hit rate for this program fragment?

There are $32/8 = 4$ blocks in the cache

Word address	Block address (Tag)	Hit/Miss	Word address	Block address (Tag)	Hit/Miss
0	0	Miss	4	2	Hit
1	0	Hit	5	2	Hit
2	1	Miss	6	3	Hit
5	2	Miss	7	3	Hit
6	3	Miss	20	10	Miss
7	3	Hit	21	10	Hit
3	1	Hit	22	11	Miss
1	0	Hit	23	11	Hit
2	1	Hit	0(唯一 Capacity)	0	Miss
3	1	Hit	1	0	Hit
Hit Rate		$13/20 = 0.65$			

降低失誤處罰時間的技術

1. **Blocking cache**：在 Cache 還未把資料搬完，任何程序都不能存取資料(*傳統 Cache 即是*)
2. **Non-blocking cache**：快取失誤時，也允許 CPU 繼續執行會存取資料快取的指令，通常被用在亂序執行(**out-of-order execution**)的處理器中，隱藏快取失誤時間。而 **Non-blocking cache** 又分：
 - (1) 失誤後命中(**hit under miss**)：在快取失誤時，允許另一個快取命中；可用其他工作來隱藏失誤延遲
 - (2) 失誤後失誤(**miss under miss**)：允許多個未完成的(**Outstanding**)快取失誤，即重疊兩個不同失誤的時間

例(70)：Non-blocking cache is used to hide the cache miss latency by using out-of-order processors. Hit under miss allows additional cache hits during a miss. Miss under miss allows multiple outstanding cache misses. (True or False)

True

例(121)：Which of the following is (are) true?

1. A non-blocking cache allows hit under miss to hide miss latency.
2. A non-blocking cache does not allow miss under hit to hide miss latency.
3. Miss under miss allows multiple outstanding cache misses.
4. A non-blocking cache allows a load instruction to access the cache if the previous load is a cache miss.

1、2、3、4

練習：The Average Memory Access Time equation (AMAT) has three components: hit time, miss rate, and miss penalty. For each of the following cache optimizations, indicate which component of the AMAT equation is improved.

1. Using a second-level cache
2. Using a direct-mapped cache
3. Using a 4-way set-associative cache
4. Using a virtually-addressed cache
5. Performing hardware pre-fetching using stream buffers
6. Using a non-blocking cache
7. Using larger blocks

1. *Miss penalty*
2. *Hit time*
3. *Miss rate*
4. *Hit time*
5. *Miss rate*
6. *Miss penalty*
7. *Miss rate*

練習：(True/False)

1. A virtual cache access time is always faster than that of a physical cache?
2. High associativity in a cache reduces compulsory misses.
3. Both DRAM and SRAM must be refreshed periodically using a dummy read/write operation.
4. A write-through cache typically requires less bus bandwidth than a write-back cache.
5. Cache performance is of less importance in faster processors because the processor speed compensates for the high memory access time.
6. Memory interleaving is a technique for reducing memory access time through increased bandwidth utilization of the data bus.

1. True (for a single level cache) since the virtual to physical address translation is avoided.
2. False. It reduces conflict misses.
3. False. Only DRAM's need to be refreshed periodically.
4. False. It is the other way round
5. False. The huge processor-memory gap implies cache performance is very critical.
6. True.

練習：If all misses are classified into one of three categories – compulsory, capacity, or conflict – which misses are likely to be reduced when a program is rewritten so as to require less memory? How about if the clock rate of the processor that the program is running on is increased? How about if the associativity of the existing cache is increased?

Less memory: fewer compulsory misses.

Increased clock rate: in general, there is no effect on the miss numbers.

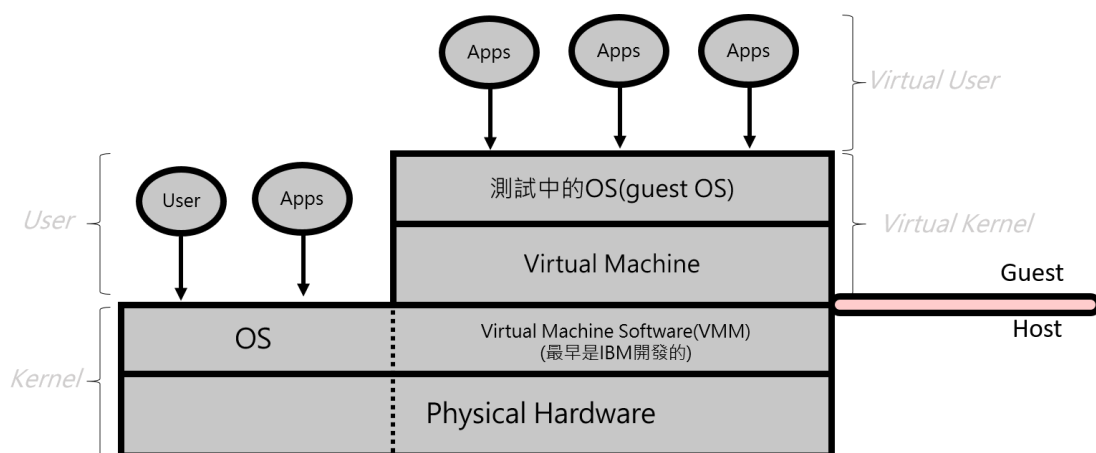
Increased associativity: fewer conflict misses.

重點十四：快取的控制設計

(已改成附錄，略過)

重點十五：虛擬機器

(一)Def：利用 Software Simulation 技術模擬出一份與底層 HW 一模一樣的功能界面之抽象化機器(abstract machine)



[名詞]：

1. Host：underlying HW System、OS
2. VMM(Virtual machine manager)或 Hypervisor：creates and managing/runs virtual machine
3. Guest

[恐]其他之英文

1. Abstract HW of a single computer into several different extension environment
2. Similar to Layered approach, but layer creates VM

(二)優點：

1. 利於測試：作為測試開發中的 OS 一個良好的負載平台，好處有：
甲、其他 user process 工作可持續運作，不需暫停
乙、萬一測試中的 OS 不穩定、掛掉、失敗了，也不會影響 host HW、OS...等其他 User process 之工作，因為只是相當於一個 User process fails 而已，不會對 System 有重大危害
2. 經濟度：同一部 host HW 上可執行多個 VM，可節省成本
3. 安全性：如果 VM 被病毒入侵，不致擴散，因為 VMs 各自獨立
4. 擴展性：可以 Freeze、Suspend、Running VM 及 clone VM
5. 合併性(Consolidation)：在 cloud computing 的時候，我們會用有限的機器建立為數很多的 VM，我們可以依 VM 上的 application 之執行負擔輕重，調動 host machines 資源作對應的支援
6. 模板化：可依需求提供不同服務
7. 移植性：可輕鬆移植

5、6、7 點皆與雲端運算有關

(三)缺點：

1. 不易製作
2. 效能較實體機來得差
3. 需要硬體支援

註：基本上與 OS 的內容相同，會放在這裡只是因為共用了『虛擬』2 個字

例(40)：

1. Virtual machines were developed in mid-1960s, but largely ignored in the domain of single-user computers in the 1980s and 1990s. Please describe why do they have recently gained popularity?
 2. Please describe the benefits that the virtual machines can provide.
 3. Please explain the requirements of a virtual machine monitor.
-
1. (1)The increasing importance of isolation and security in modern systems.
(2)The failures in security and reliability of standard operating system.
(3)The sharing of a single computer among many unrelated user.
(4)The dramatic increases in raw speed of processors over the decades, which makes the overhead of VMs more acceptable.

2. 優點：

(1)利於測試：作為測試開發中的 OS 一個良好的負載平台，好處有：

i.其他 user process 工作可持續運作，不需暫停

ii.萬一測試中的 OS 不穩定、掛掉、失敗了，也不會影響 host HW、OS...等其他 User process 之工作，因為只是相當於一個 User process fails 而已，不會對 System 有重大危害

(2)經濟度：同一部 host HW 上可執行多個 VM，可節省成本

(3)安全性：如果 VM 被病毒入侵，不致擴散，因為 VMs 各自獨立

(4)擴展性：可以 Freeze、Suspend、Running VM 及 clone VM

(5)合併性(Consolidation)：在 cloud computing 的時候，我們會用有限的機器建立為數很多的 VM，我們可以依 VM 上的 application 之執行負擔輕重，調動 host machines 資源作對應的支援

(6)模板化：可依需求提供不同服務

(7)移植性：可輕鬆移植

3. (1)Guest software should behave on a VM exactly as if it were running on the native hardware.

(2)Guest software should not be able to change allocation of real system resources directly.

例(2)：Please explain the term: virtual machine monitor in English. Please spend no more than 100 English words.

The software that supports virtual machines is called Virtual machine monitor. (It presents a software interface to guest software and must isolate the state of guests from each other and protect itself from guest software.)

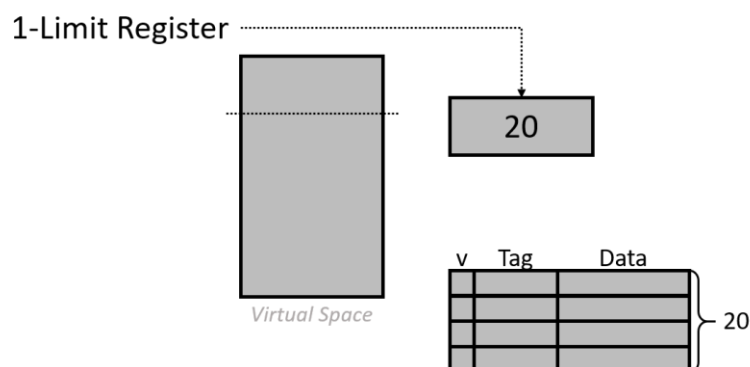
減少分頁表的大小

例：假設有一 32 位元的虛擬位址，每頁為 4KB 分頁表的每個項目為 4 個位組，我們可以算出分頁表的總大小為：

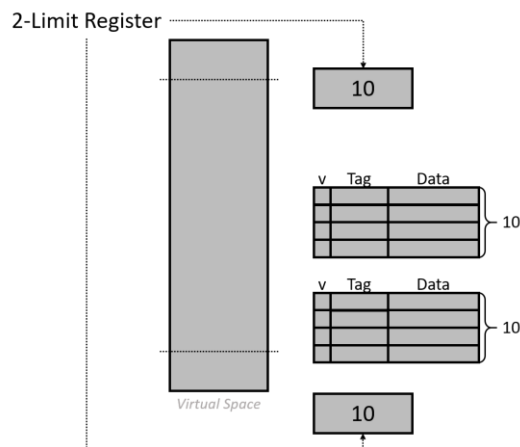
$$2^{32}/4K = 2^{20} = 1 \text{ Mega entries} ; \text{Page Table size} = 1M * 4 \text{ bytes} = 4MB$$

由上可知，分頁表會佔用掉一大部分的 Memory，故以下介紹 5 種方式，可降低所需要的總儲存空間、與最小化分頁：

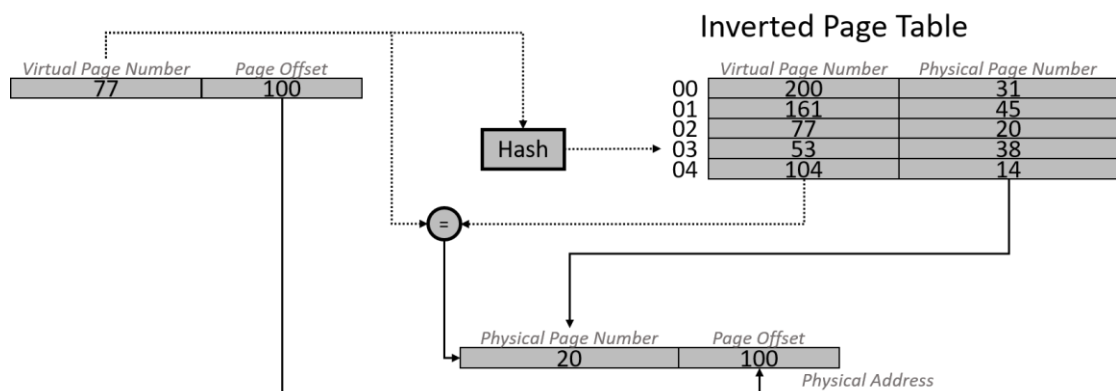
1. 1 個限制暫存器(Limit Register)：大部分程式所使用到的 page 不會超過 20 個，因此預設便是 20 個 page，若有多出來的，再以動態加入即可



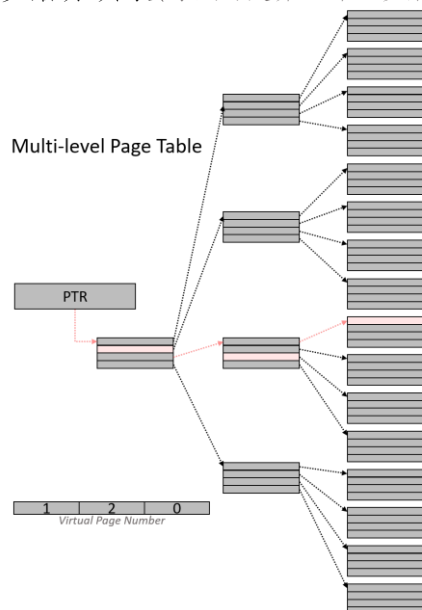
- 2 個限制暫存器：分別使用 **stack** 與 **heap** 存入兩張分開分頁表(MIPS 使用此法)



- 使用 **hashing/反轉式分頁表**(OS 為分開的 2 種方式)：確實有減少分頁表大小



- 多層分頁表(考試時是算全部『多層』的大小)



- 分頁也可以被分頁：架構相同，但平移往下一層至硬碟中

練習：The memory architecture of a machine X is summarized in the Table.

Virtual Address	54 bits
Page Size	16Kbytes
Page Table Entries Size	4 bytes

1. Assume that there are 8 bits reserved for the operating system function (protection, replacement, valid, modified, ...) other than required by the hardware translation algorithm. Derive the largest physical memory size (in bytes) allowed by this PTE format. Make sure you consider all the fields required by the translation algorithm.
2. How large (in bytes) is the page table?
3. Assuming 1 application exists in the system and the maximum physical memory is devoted to the process, how much physical space (in bytes) is there for the application's data and code.

1. *Physical page number = $32 - 8 = 24$ bits.
The largest physical memory size = $2^{24} * 16\text{Kbytes} = 256\text{ GB}$*
2. *The virtual page number has $54 - 14 = 40$ bits. The number table entries are 2^{40} . Each PTE has 4 bytes. So the total size of the page table is 2^{42} bytes which is 4 terabytes.*
3. *The application's page table has an entry for every physical page that exists on the system, which means the page table size is $2^{24} * 4$ bytes. This leaves the remaining physical space to the process: $2^{24} * 16\text{K} - 2^{26} \text{ bytes} = 2^{38} - 2^{26} \text{ bytes}$.*

練習：Consider a machine with 32-bit virtual addresses, 32-bit physical addresses, and a 4KB page size. Consider a two-level page table system where each table occupies one full page. Assume each page table entry is 32 bits long. To map the full virtual address space, how much memory will be used by the page table?

*The number of entries in the page table = $4\text{KB} / 4\text{B} = 1024$
Amount of memory used = $4\text{KB} + 1024 * 4\text{KB} = 4.0001\text{MB}$*